

Machine Learning: A Field Guide for the Perpetually Confused

From “What Is a Gradient” to “Why Is My Loss Exploding”

Erdan Mu Alan

University of Pennsylvania

Contents

I	Foundations	1
1	The Learning Problem	2
1.1	What Is Machine Learning?	2
1.2	The Statistical Learning Setup	2
2	Probability and Statistics	4
2.1	Random Variables	4
2.2	Expectation and Variance	4
2.3	Common Distributions	5
2.4	Maximum Likelihood Estimation	5
2.5	Multiple Random Variables	7
2.6	Multivariate Gaussian Distribution	8
3	Linear Algebra	9
3.1	Vectors and Matrices	9
3.2	Matrix Properties	10
3.3	Eigendecomposition	10
3.4	Positive Semidefinite Matrices	11
3.5	Singular Value Decomposition	11
4	Multivariate Calculus	12
4.1	Gradient	12
4.2	Jacobian and Hessian	12
4.3	Chain Rule and Matrix Calculus	13
5	Convex Optimization	14
5.1	Convex Sets and Functions	14
5.2	Verifying Convexity	15
5.3	Gradient Descent	15
5.4	Stochastic Gradient Descent	16
6	Model Evaluation	17
6.1	The Bias-Variance Decomposition	17
6.2	Cross-Validation	18
6.3	Classification Metrics	18

II	Supervised Learning	19
7	Linear Regression	20
7.1	The Linear Model	20
7.2	Loss Functions	20
7.3	Ordinary Least Squares	21
7.4	Probabilistic Interpretation	21
7.5	Collinearity	22
8	Logistic Regression	23
8.1	From Regression to Classification	23
8.2	Log-Odds and the Logit Link	23
8.3	Learning via Maximum Likelihood	24
8.4	Gradient and Gradient Descent	24
8.5	Multiclass: Softmax Regression	24
8.6	Limitations	25
9	Regularization	26
9.1	The Overfitting Problem	26
9.2	Ridge Regression (ℓ_2 Regularization)	26
9.3	Lasso (ℓ_1 Regularization)	27
9.4	Elastic Net	27
9.5	Regularization for Logistic Regression	28
9.6	Effect on Bias and Variance	28
10	Support Vector Machines	29
10.1	Hard-Margin SVM	29
10.2	Dual Formulation	29
10.3	Soft-Margin SVM and Hinge Loss	30
10.4	The Kernel Trick	30
11	K-Nearest Neighbors	32
11.1	The Algorithm	32
11.2	Distance Metrics	32
11.3	Choosing K and the Bias-Variance Tradeoff	33
11.4	Curse of Dimensionality	33
12	Naïve Bayes	34
12.1	Generative vs. Discriminative Classifiers	34
12.2	Bayes' Rule for Classification	34
12.3	The Naïve Bayes Assumption	35
12.4	Parameter Estimation	35
12.5	Laplace Smoothing	35
13	Decision Trees and Ensembles	36
13.1	Decision Trees	36
13.2	Splitting Criteria	36
13.3	Overfitting and Pruning	37
13.4	Random Forests	37
13.5	Boosting	37

III	Unsupervised Learning	39
14	Clustering	40
14.1	The Unsupervised Setting	40
14.2	K-Means Algorithm	40
14.3	Convergence	41
14.4	Initialisation: K-Means++	41
14.5	Connection to Gaussian Mixture Models	41
14.6	Limitations of K-Means	42
15	Dimensionality Reduction	43
15.1	Motivation	43
15.2	Principal Component Analysis	43
15.3	PCA via Singular Value Decomposition	44
15.4	Autoencoders: Non-linear PCA	44
15.5	Limitations of PCA	45
16	Learning Paradigms	46
16.1	A Taxonomy of Learning	46
16.2	Transfer Learning	47
16.3	Reinforcement Learning	47
IV	Deep Learning	48
17	Neural Network Foundations	49
17.1	The Single Neuron	49
17.2	Multi-Layer Perceptrons	49
17.3	Activation Functions	50
18	Training Deep Networks	51
18.1	Backpropagation	51
18.2	Weight Initialisation	51
18.3	Batch Normalisation	52
18.4	Dropout	52
18.5	Optimisation	52
18.6	Learning Rate Schedules	52
18.7	Data Augmentation and Mixup	53
19	Convolutional Neural Networks	54
19.1	The Convolution Operation	54
19.2	Pooling	54
19.3	Typical CNN Architecture	54
19.4	Classic Architectures	55
20	Object Detection and Segmentation	56
20.1	From Classification to Localisation	56
20.2	Two-Stage Detectors: Faster R-CNN	56
20.3	One-Stage Detectors: YOLO	56
20.4	Semantic Segmentation	56

21 Recurrent Neural Networks	57
21.1 Sequence Modelling	57
21.2 Vanilla RNN	57
21.3 Backpropagation Through Time	58
21.4 Long Short-Term Memory	58
21.5 Bidirectional and Multi-Layer RNNs	59
22 Attention and Transformers	60
22.1 Scaled Dot-Product Attention	60
22.2 Multi-Head Attention	60
22.3 The Transformer Architecture	61
22.4 Pre-Trained Transformers	61
V Generative Models	63
23 Variational Autoencoders	64
23.1 The Generative Modelling Goal	64
23.2 Evidence Lower Bound	64
23.3 The Reparametrisation Trick	64
24 Generative Adversarial Networks	66
24.1 The GAN Minimax Game	66
24.2 Practical Variants	66
25 Diffusion Models	67
25.1 Forward Diffusion Process	67
25.2 Reverse Process and Training	67
VI Large Language Models	68
26 Language Models and Pretraining	69
26.1 Language Modelling	69
26.2 Subword Tokenisation	69
26.3 Transformer LM Architectures	69
26.4 Scaling Laws	70
27 Instruction Finetuning and Alignment	71
27.1 Supervised Fine-Tuning	71
27.2 Reinforcement Learning from Human Feedback	71
27.3 Direct Preference Optimisation	71
28 In-Context Learning and Prompting	72
28.1 In-Context Learning	72
28.2 Chain-of-Thought Prompting	72
29 Decoding Strategies	73
29.1 Deterministic Decoding	73
29.2 Stochastic Decoding	73

30 Retrieval-Augmented Generation	74
30.1 Motivation	74
31 LLM Agents	75
31.1 Agent Architecture	75
31.2 Memory Types	75
31.3 Multi-Agent Systems	75
VII ML Engineering	76
32 Transfer Learning and Fine-Tuning	77
32.1 Transfer Learning	77
32.2 Parameter-Efficient Fine-Tuning	77
33 Hyperparameter Optimization	78
33.1 Problem Statement	78
33.2 Search Strategies	78
33.3 Multi-Fidelity Methods	78
34 Distributed Training	79
34.1 Parallelism Strategies	79
34.2 Gradient Synchronisation	79
34.3 Mixed-Precision Training	79
35 AutoML and Knowledge Distillation	80
35.1 Neural Architecture Search	80
35.2 Knowledge Distillation	80
VIII Reinforcement Learning	81
36 Multi-Armed Bandits	82
36.1 The Bandit Problem	82
36.2 Exploration–Exploitation Strategies	82
37 Markov Decision Processes	83
37.1 MDP Framework	83
37.2 Bellman Equations	83
38 Dynamic Programming	85
38.1 Policy Evaluation and Improvement	85
39 Temporal Difference Learning	86
39.1 TD Prediction	86
39.2 Q-Learning and SARSA	86
39.3 Deep Q-Network	87
40 Policy Gradient Methods	88
40.1 Policy Gradient Theorem	88
40.2 Actor-Critic	88
40.3 Proximal Policy Optimisation	88

Part I

Foundations

Chapter 1

The Learning Problem

1.1 What Is Machine Learning?

Definition 1.1.1: Machine Learning

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E . (*Mitchell, 1997*)

Every ML system sits at the intersection of three components:

- **Model** — the family of functions $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ parameterized by θ .
- **Data** — observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ used to constrain θ .
- **Optimization** — the algorithm that searches for the best θ given $\mathcal{D}$.

The three learning paradigms differ in the structure of data:

Supervised learning Labeled pairs $(\mathbf{x}, y)$. Goal: predict y from $\mathbf{x}$.

Unsupervised learning Unlabeled observations $\{\mathbf{x}_i\}$. Goal: discover structure.

Reinforcement learning Agent–environment interaction with reward signals. Goal: find a policy maximizing cumulative reward.

1.2 The Statistical Learning Setup

Definition 1.2.1: Hypothesis Class

A **hypothesis class** $\mathcal{H}$ is a set of candidate functions $h : \mathcal{X} \rightarrow \mathcal{Y}$. Learning means searching for the best $h \in \mathcal{H}$.

Definition 1.2.2: Loss Function

A **loss function** $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ quantifies the cost of predicting $\hat{y}$ when the truth is y . Common choices:

Name	Formula	Use case
Squared loss	$\ell(\hat{y}, y) = (\hat{y} - y)^2$	Regression
0-1 loss	$\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$	Classification
Cross-entropy	$\ell(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$	Binary classification

Definition 1.2.3: Risk and Empirical Risk Minimization

The **true risk** (generalization error) of h is the expected loss under the data distribution:

$$R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[\ell(h(\mathbf{x}), y)].$$

Since $\mathcal{D}$ is unknown, we minimize the **empirical risk** on the training set:

$$\hat{R}(h) = \frac{1}{N} \sum_{i=1}^N \ell(h(\mathbf{x}_i), y_i).$$

Empirical Risk Minimization (ERM): $\hat{h} = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$.

The difference $R(\hat{h}) - \hat{R}(\hat{h})$ is the **generalization gap**. A model **overfits** if this gap is large; it **underfits** if both R and $\hat{R}$ are large. The capacity of $\mathcal{H}$ controls this tradeoff.

Remark.

Data is divided into three non-overlapping sets:

Training set Used to minimize $\hat{R}(h)$.

Validation set Used for model selection and hyperparameter tuning.

Test set Used once, at the very end, as an unbiased estimate of $R(\hat{h})$.

Never touch the test set until final evaluation.

Chapter 2

Probability and Statistics

2.1 Random Variables

Definition 2.1.1: Random Variable

A **random variable** X is a function mapping outcomes from a probability space to $\mathbb{R}$. We write $P(X = x)$ or $P(X \leq x)$ for event probabilities.

For a **discrete** RV: the **PMF** $p_X(x) = P(X = x)$ satisfies $\sum_x p_X(x) = 1$; the **CDF** $F_X(x) = \sum_{x' \leq x} p_X(x')$.

For a **continuous** RV: the **PDF** $f_X(x) = \frac{dF_X(x)}{dx}$ satisfies $\int f_X(x) dx = 1$; note $f_X(x) \neq P(X = x)$. The **CDF** $F_X(x) = \int_{-\infty}^x f_X(t) dt$.

2.2 Expectation and Variance

Definition 2.2.1: Expectation

$$E[X] = \begin{cases} \sum x p_X(x) & \text{(discrete)} \\ \int_{-\infty}^{\infty} x f_X(x) dx & \text{(continuous)} \end{cases}$$

More generally, $E[g(X)] = \int g(x) f_X(x) dx$.

Proposition 2.2.2

Linearity of expectation: for any constants a, b and random variables X, Y :

$$E[aX + b] = aE[X] + b, \quad E[X + Y] = E[X] + E[Y].$$

Linearity holds regardless of whether X and Y are independent.

Definition 2.2.3: Variance

$$\text{Var}[X] = E[(X - E[X])^2] = E[X^2] - (E[X])^2.$$

The **standard deviation** is $\sigma_X = \sqrt{\text{Var}[X]}$.

Proposition 2.2.4

Key variance identities: $\text{Var}[a] = 0$, $\text{Var}[aX] = a^2 \text{Var}[X]$, and

$$\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] + 2 \text{Cov}[X, Y].$$

If X, Y are independent, $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y]$.

2.3 Common Distributions

Distribution	PMF / PDF	Mean	Variance
Bernoulli(p)	$p^x(1-p)^{1-x}$, $x \in \{0, 1\}$	p	$p(1-p)$
Binomial(n, p)	$\binom{n}{x}p^x(1-p)^{n-x}$, $x \in \{0, \dots, n\}$	np	$np(1-p)$
Geometric(p)	$p(1-p)^{k-1}$, $k = 1, 2, \dots$	$\frac{1}{p}$	$\frac{1-p}{p^2}$
Poisson(λ)	$\frac{e^{-\lambda}\lambda^k}{k!}$, $k = 0, 1, \dots$	λ	λ
Uniform(a, b)	$\frac{1}{b-a}$ for $x \in (a, b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
$\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$	μ	σ^2
Exponential(λ)	$\lambda e^{-\lambda x}$, $x \geq 0$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$

2.4 Maximum Likelihood Estimation

Given data $\{x_i\}_{i=1}^N$ assumed i.i.d. from $p(x; \theta)$, MLE finds the parameter making observations most probable.

Definition 2.4.1: Likelihood, Log-Likelihood, and MLE

$$L(\theta) = \prod_{i=1}^N p(x_i; \theta), \quad \ell(\theta) = \log L(\theta) = \sum_{i=1}^N \log p(x_i; \theta).$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \ell(\theta).$$

The log transform converts the product into a sum and has the same maximizer since log is monotone.

Theorem 2.4.2: MLE for the Gaussian Distribution

Given N i.i.d. samples from $\mathcal{N}(\mu, \sigma^2)$, the MLEs are:

$$\mu_{\text{ML}} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma_{\text{ML}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\text{ML}})^2.$$

Proof

The log-likelihood expands as:

$$\begin{aligned}\ell(\mu, \sigma^2) &= \sum_{i=1}^N \log \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right) \\ &= -N \log \sigma - \frac{N}{2} \log(2\pi) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2.\end{aligned}$$

Estimating μ : Set $\frac{\partial \ell}{\partial \mu} = 0$:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^N (x_i - \mu) = 0 \implies \mu_{\text{ML}} = \frac{1}{N} \sum_{i=1}^N x_i.$$

Estimating σ^2 : Let $\tau = \sigma^2$ and set $\frac{\partial \ell}{\partial \tau} = 0$:

$$\frac{\partial \ell}{\partial \tau} = -\frac{N}{2\tau} + \frac{1}{2\tau^2} \sum_{i=1}^N (x_i - \mu)^2 = 0 \implies \sigma_{\text{ML}}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{\text{ML}})^2.$$

□

Theorem 2.4.3: MLE Variance Estimator is Biased

$E[\sigma_{\text{ML}}^2] = \frac{N-1}{N} \sigma^2 \neq \sigma^2$. The unbiased estimator is the **sample variance** $\hat{\sigma}^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$.

Proof

Write $\bar{x} = \mu_{\text{ML}}$. Use the identity $(x_i - \bar{x}) = (x_i - \mu) - (\bar{x} - \mu)$:

$$\begin{aligned}\sum_{i=1}^N (x_i - \bar{x})^2 &= \sum_{i=1}^N [(x_i - \mu) - (\bar{x} - \mu)]^2 \\ &= \sum_{i=1}^N (x_i - \mu)^2 - 2(\bar{x} - \mu) \underbrace{\sum_{i=1}^N (x_i - \mu)}_{= N(\bar{x} - \mu)} + N(\bar{x} - \mu)^2 \\ &= \sum_{i=1}^N (x_i - \mu)^2 - N(\bar{x} - \mu)^2.\end{aligned}$$

Taking expectations (with $E[(x_i - \mu)^2] = \sigma^2$ and $\text{Var}[\bar{x}] = \sigma^2/N$):

$$E[\sigma_{\text{ML}}^2] = \frac{1}{N} \left(N\sigma^2 - N \cdot \frac{\sigma^2}{N} \right) = \sigma^2 - \frac{\sigma^2}{N} = \frac{N-1}{N} \sigma^2.$$

□

Theorem 2.4.4: Strong Law of Large Numbers

Let $X_1, X_2, \dots$ be i.i.d. with $E[X_i] = \mu < \infty$. Then, with probability 1:

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{\text{a.s.}} \mu \quad \text{as } n \rightarrow \infty.$$

This justifies using the empirical mean as a proxy for the true mean, and more broadly underpins why ERM works.

2.5 Multiple Random Variables**Definition 2.5.1: Joint, Marginal, and Conditional Distributions**

For two random variables X and Y :

Joint PMF (discrete):	$p_{XY}(x, y) = P(X = x, Y = y)$
Marginal PMF:	$p_X(x) = \sum_y p_{XY}(x, y)$
Joint PDF (continuous):	$f_{XY}(x, y) = \frac{\partial^2 F_{XY}(x, y)}{\partial x \partial y}$
Marginal PDF:	$f_X(x) = \int f_{XY}(x, y) dy$
Conditional PDF:	$f_{Y X=x}(y) = \frac{f_{XY}(x, y)}{f_X(x)}$

Theorem 2.5.2: Bayes' Theorem

$$P(B | A) = \frac{P(A | B) P(B)}{P(A)}.$$

For continuous random variables:

$$f_{Y|X=x}(y) = \frac{f_{X|Y=y}(x) f_Y(y)}{f_X(x)}.$$

Definition 2.5.3: Independence

X and Y are **independent** if $f_{XY}(x, y) = f_X(x) f_Y(y)$ (equivalently, $p_{XY}(x, y) = p_X(x) p_Y(y)$ for discrete). Independence implies $E[XY] = E[X]E[Y]$.

Definition 2.5.4: Covariance and Pearson Correlation

$$\text{Cov}[X, Y] = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y].$$

$$\rho_{X,Y} = \frac{\text{Cov}[X, Y]}{\sigma_X \sigma_Y} \in [-1, 1].$$

Independence implies $\text{Cov}[X, Y] = 0$, but the converse is false in general. ρ measures linear dependence: $|\rho| = 1$ iff $Y = aX + b$ for some $a \neq 0$.

For a random vector $\mathbf{X} = (X_1, \dots, X_n)^\top$, the **covariance matrix** is:

$$\Sigma = E[(\mathbf{X} - E[\mathbf{X}])(\mathbf{X} - E[\mathbf{X}])^\top], \quad \Sigma_{ij} = \text{Cov}[X_i, X_j].$$

Σ is always symmetric and positive semidefinite.

The **sample** counterparts from observations $\{(x_i, y_i)\}_{i=1}^N$:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \hat{\sigma}^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2, \quad \hat{\sigma}_{XY}^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y}).$$

2.6 Multivariate Gaussian Distribution

Definition 2.6.1: Multivariate Gaussian

A random vector $\mathbf{X} \in \mathbb{R}^n$ follows $\mathcal{N}(\mu, \Sigma)$ with mean $\mu \in \mathbb{R}^n$ and PSD covariance $\Sigma \in \mathbb{R}^{n \times n}$ if:

$$f(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)\right).$$

The quantity $(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)$ is the squared **Mahalanobis distance**, which reduces to $\frac{\|\mathbf{x} - \mu\|^2}{\sigma^2}$ when $\Sigma = \sigma^2 I$.

Chapter 3

Linear Algebra

The structure of this chapter follows a hierarchy from general to structured:

General matrices $\supset$ Diagonalizable $\supset$ Symmetric $\supset$ PSD.

More structure yields nicer properties that ML heavily exploits.

3.1 Vectors and Matrices

We work with real-valued matrices throughout. Vectors are column vectors by default: $\mathbf{x} \in \mathbb{R}^n$ means $\mathbf{x} \in \mathbb{R}^{n \times 1}$.

Definition 3.1.1: Basic Operations

For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $A \in \mathbb{R}^{m \times n}$:

- **Inner product:** $\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i \in \mathbb{R}$
- **Outer product:** $\mathbf{x} \mathbf{y}^\top \in \mathbb{R}^{n \times n}$, with $(\mathbf{x} \mathbf{y}^\top)_{ij} = x_i y_j$
- **Matrix–vector product:** $(A \mathbf{x})_i = \sum_{j=1}^n A_{ij} x_j$
- **Transpose:** $(A^\top)_{ij} = A_{ji}$; $(AB)^\top = B^\top A^\top$

Definition 3.1.2: Vector Norms

The ℓ_p norm of $\mathbf{x} \in \mathbb{R}^n$: $\|\mathbf{x}\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$.

- $\|\mathbf{x}\|_1 = \sum_i |x_i|$ (sparsity-inducing; used in Lasso)
- $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$ (Euclidean; default $\|\mathbf{x}\|$)
- $\|\mathbf{x}\|_\infty = \max_i |x_i|$

3.2 Matrix Properties

Theorem 3.2.1: Equivalences for Invertible Matrices

For $A \in \mathbb{R}^{n \times n}$, the following are equivalent:

1. A is invertible (A^{-1} exists with $AA^{-1} = I$).
2. $\det(A) \neq 0$.
3. $\text{rank}(A) = n$ (full rank).
4. $A\mathbf{x} = \mathbf{0}$ implies $\mathbf{x} = \mathbf{0}$ (trivial null space).
5. The columns (and rows) of A are linearly independent.

Remark.

The **trace** $\text{tr}(A) = \sum_i A_{ii}$ equals the sum of eigenvalues; the **determinant** $\det(A)$ equals their product. Both are invariant under similarity transforms: $\text{tr}(P^{-1}AP) = \text{tr}(A)$.

3.3 Eigendecomposition

Definition 3.3.1: Eigenvalue and Eigenvector

A scalar $\lambda \in \mathbb{R}$ and non-zero vector $\mathbf{v} \in \mathbb{R}^n$ satisfy

$$A\mathbf{v} = \lambda\mathbf{v}$$

if λ is an **eigenvalue** and $\mathbf{v}$ the corresponding **eigenvector** of $A \in \mathbb{R}^{n \times n}$. The eigenvalues are the roots of the **characteristic polynomial** $\det(A - \lambda I) = 0$.

Theorem 3.3.2: Spectral Theorem for Symmetric Matrices

If $A \in \mathbb{R}^{n \times n}$ is symmetric ($A = A^\top$), then:

1. All eigenvalues are real.
2. Eigenvectors for distinct eigenvalues are orthogonal.
3. A is **orthogonally diagonalizable**: $A = Q\Lambda Q^\top$ where Q is orthogonal ($Q^\top Q = I$), its columns are eigenvectors, and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

3.4 Positive Semidefinite Matrices

Definition 3.4.1: Positive Semidefinite (PSD)

A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is **positive semidefinite** ($A \succeq 0$) if:

$$\mathbf{x}^\top A \mathbf{x} \geq 0 \quad \text{for all } \mathbf{x} \in \mathbb{R}^n.$$

It is **positive definite** ($A \succ 0$) if equality holds only for $\mathbf{x} = \mathbf{0}$.

Theorem 3.4.2: PSD Equivalences

For a symmetric $A \in \mathbb{R}^{n \times n}$, the following are equivalent:

1. $A \succeq 0$ (by definition).
2. All eigenvalues $\lambda_i \geq 0$.
3. $A = B^\top B$ for some matrix B .

Remark.

PSD matrices arise as covariance matrices ($\Sigma \succeq 0$ always), and as Hessians of convex functions. The 2nd-order convexity test ($\nabla^2 f \succeq 0$) is precisely a PSD check.

3.5 Singular Value Decomposition

Theorem 3.5.1: Singular Value Decomposition

Every matrix $A \in \mathbb{R}^{m \times n}$ of rank r admits:

$$A = U \Sigma V^\top,$$

where $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ are orthogonal, and $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with entries $\sigma_1 \geq \dots \geq \sigma_r > 0 = \sigma_{r+1} = \dots$ called **singular values**.

The columns of U are eigenvectors of $AA^\top$; columns of V are eigenvectors of $A^\top A$; $\sigma_i = \sqrt{\lambda_i(A^\top A)}$. SVD is the computational foundation of PCA.

Chapter 4

Multivariate Calculus

4.1 Gradient

Definition 4.1.1: Gradient

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** is the vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \in \mathbb{R}^n.$$

$\nabla f(\mathbf{x})$ points in the direction of *steepest ascent*; $-\nabla f(\mathbf{x})$ toward steepest descent.

4.2 Jacobian and Hessian

Definition 4.2.1: Jacobian

For $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the **Jacobian** $J \in \mathbb{R}^{m \times n}$ is:

$$J_{ij} = \frac{\partial f_i}{\partial x_j}, \quad J = \begin{pmatrix} \nabla f_1^\top \\ \vdots \\ \nabla f_m^\top \end{pmatrix}.$$

Definition 4.2.2: Hessian

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **Hessian** $H = \nabla^2 f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ is the matrix of second partial derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}, \quad \nabla^2 f(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}.$$

By Schwarz's theorem, H is symmetric whenever mixed partials are continuous.

4.3 Chain Rule and Matrix Calculus

Proposition 4.3.1

Univariate: $\frac{d}{dt}f(g(t)) = f'(g(t)) \cdot g'(t)$.

Multivariate: If $\mathbf{x} = \mathbf{g}(\mathbf{t})$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\frac{\partial f}{\partial t_k} = \sum_{i=1}^n \frac{\partial f}{\partial x_i} \frac{\partial x_i}{\partial t_k}, \quad \nabla_{\mathbf{t}} f = J_{\mathbf{g}}^{\top} \nabla_{\mathbf{x}} f.$$

Fact 4.3.2

Useful matrix calculus identities (where A is a constant matrix):

$$\begin{aligned} \nabla_{\mathbf{x}} (A\mathbf{x}) &= A^{\top} \\ \nabla_{\mathbf{x}} (\mathbf{x}^{\top} A \mathbf{x}) &= (A + A^{\top})\mathbf{x} = 2A\mathbf{x} \quad (\text{if } A \text{ symmetric}) \end{aligned}$$

The second identity is used directly in the normal equations for linear regression.

Chapter 5

Convex Optimization

5.1 Convex Sets and Functions

Definition 5.1.1: Convex Set

$\mathcal{C} \subseteq \mathbb{R}^d$ is **convex** if for any $\mathbf{x}, \mathbf{y} \in \mathcal{C}$ and $\theta \in [0, 1]$:

$$\theta \mathbf{x} + (1 - \theta) \mathbf{y} \in \mathcal{C}.$$

Examples: $\mathbb{R}^d$; affine subspaces $\{\mathbf{x} \mid A\mathbf{x} = \mathbf{b}\}$; polyhedra $\{\mathbf{x} \mid A\mathbf{x} \leq \mathbf{b}\}$. Intersections of convex sets are convex.

Definition 5.1.2: Convex Function

$f : \mathcal{D} \rightarrow \mathbb{R}$ (with convex $\mathcal{D}$) is **convex** if for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{D}$ and $\lambda \in [0, 1]$:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2).$$

f is **strictly convex** if strict inequality holds for $\mathbf{x}_1 \neq \mathbf{x}_2$, $\lambda \in (0, 1)$. f is **concave** if $-f$ is convex.

Theorem 5.1.3: Global Optimality of Critical Points

If f is convex and $\mathbf{x}^*$ satisfies $\nabla f(\mathbf{x}^*) = \mathbf{0}$ (or the gradient does not exist there), then $\mathbf{x}^*$ is a **global minimum** of f .

This is the central reason ML prefers convex losses: any optimizer that finds a critical point has found the global solution.

5.2 Verifying Convexity

Theorem 5.2.1: Three Convexity Tests — Scalar

For $f : \mathbb{R} \rightarrow \mathbb{R}$, each of the following is equivalent to convexity:

1. **0th-order:** $f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$ (definition).
2. **1st-order (tangent line below):** $f(x_2) \geq f(x_1) + f'(x_1)(x_2 - x_1)$ for all x_1, x_2 .
3. **2nd-order (non-negative curvature):** $f''(x) \geq 0$ for all x .

Theorem 5.2.2: Three Convexity Tests — Multivariate

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with convex domain, each of the following is equivalent to convexity:

1. **0th-order:** $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$.
2. **1st-order (tangent hyperplane below):** $f(\mathbf{y}) \geq f(\mathbf{x}) + (\nabla f(\mathbf{x}))^\top (\mathbf{y} - \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$.
3. **2nd-order (Hessian PSD):** $\nabla_{\mathbf{x}}^2 f(\mathbf{x}) \succeq 0$ for all $\mathbf{x}$.

Equivalently: f is convex iff its restriction to every line is convex.

Example 5.2.1

$f(\mathbf{x}) = 2x_1^2 + x_2^2$ has $H = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}$. Eigenvalues $4, 2 > 0$, so $H \succ 0$ and f is strictly convex.

$f(\mathbf{x}) = 2x_1^2 - x_2^2$ has $H = \begin{pmatrix} 4 & 0 \\ 0 & -2 \end{pmatrix}$. Eigenvalue $-2 < 0$, so $H \not\succeq 0$ and f is not convex.

$f(\mathbf{x}) = x_1 + 2x_1^2 + x_2^2 - x_1x_2 - 2$ has $H = \begin{pmatrix} 4 & -1 \\ -1 & 2 \end{pmatrix}$. Eigenvalues $3 \pm \sqrt{2} > 0$, so f is strictly convex.

5.3 Gradient Descent

Definition 5.3.1: Gradient Descent

To minimize $f(\mathbf{w})$, initialize $\mathbf{w}_0$ and iterate:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t), \quad t = 0, 1, 2, \dots$$

until $\|\nabla f(\mathbf{w}_t)\| < \varepsilon$ or max iterations reached. The scalar $\eta > 0$ is the **learning rate**.

Remark.

Choosing η : too small $\Rightarrow$ slow convergence; too large $\Rightarrow$ oscillation or divergence. For an L -smooth function (Lipschitz gradient constant L), the theoretically safe step size is $\eta = 1/L$.

Theorem 5.3.2: GD Convergence for Convex Functions

Let f be convex and L -smooth ($\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$). With step size $\eta = 1/L$ and global minimizer $\mathbf{w}^*$:

$$f(\mathbf{w}_t) - f(\mathbf{w}^*) \leq \frac{L\|\mathbf{w}_0 - \mathbf{w}^*\|^2}{2t} = O\left(\frac{1}{t}\right).$$

For μ -strongly convex f , the rate improves to $O\left(\left(1 - \frac{\mu}{L}\right)^t\right)$ (linear/geometric convergence).

5.4 Stochastic Gradient Descent

When $f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \ell_i(\mathbf{w})$ (ERM), computing ∇f requires $O(N)$ evaluations per step.

Definition 5.4.1: SGD and Mini-Batch SGD

SGD samples a single index $i_t \sim \text{Uniform}\{1, \dots, N\}$ per step:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla \ell_{i_t}(\mathbf{w}_t).$$

Mini-batch SGD averages over a batch $\mathcal{B} \subset \{1, \dots, N\}$ of size B :

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}_t).$$

The key property: $E_{i_t}[\nabla \ell_{i_t}(\mathbf{w})] = \nabla f(\mathbf{w})$. SGD uses an **unbiased estimator** of the true gradient, enabling progress while spending only $O(B)$ per update.

Chapter 6

Model Evaluation

6.1 The Bias-Variance Decomposition

Suppose the true relationship is $y = f(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$. Let $\hat{f}$ be a model trained on a random dataset $\mathcal{D}$.

Theorem 6.1.1: Bias-Variance-Noise Decomposition

The expected squared error at a fixed point $\mathbf{x}$ decomposes as:

$$E_{\mathcal{D}, \varepsilon} \left[(y - \hat{f}(\mathbf{x}))^2 \right] = \underbrace{\left(f(\mathbf{x}) - E_{\mathcal{D}}[\hat{f}(\mathbf{x})] \right)^2}_{\text{Bias}^2} + \underbrace{E_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}) - E_{\mathcal{D}}[\hat{f}(\mathbf{x})] \right)^2 \right]}_{\text{Variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{Noise}}.$$

Proof

Let $\bar{f} = E_{\mathcal{D}}[\hat{f}(\mathbf{x})]$. First isolate the noise:

$$\begin{aligned} E \left[(y - \hat{f})^2 \right] &= E \left[(f + \varepsilon - \hat{f})^2 \right] \\ &= E \left[(f - \hat{f})^2 \right] + 2 \underbrace{E[\varepsilon]}_{=0} E[f - \hat{f}] + E[\varepsilon^2] \\ &= E \left[(f - \hat{f})^2 \right] + \sigma_\varepsilon^2. \end{aligned}$$

Now add and subtract $\bar{f}$:

$$\begin{aligned} E \left[(f - \hat{f})^2 \right] &= E \left[(f - \bar{f} + \bar{f} - \hat{f})^2 \right] \\ &= (f - \bar{f})^2 + 2(f - \bar{f}) \underbrace{E[\bar{f} - \hat{f}]}_{=0} + E \left[(\bar{f} - \hat{f})^2 \right] \\ &= \text{Bias}^2[\hat{f}] + \text{Var}[\hat{f}]. \quad \square \end{aligned}$$

□

The decomposition reveals a fundamental tradeoff:

- **High bias, low variance** (underfitting): model too simple; increase capacity.
- **Low bias, high variance** (overfitting): model too complex; regularize or collect more data.

- **Noise** σ_ε^2 : irreducible; no model can eliminate it.

6.2 Cross-Validation

Definition 6.2.1: k -Fold Cross-Validation

Partition training data into k equal folds. For each fold $j = 1, \dots, k$: train on the remaining $k - 1$ folds, evaluate on fold j . Report the average metric over all k folds.

k -fold CV provides a lower-variance estimate of $R(h)$ than a single split, especially for small N . Common choices: $k = 5$ or $k = 10$. Special case $k = N$: **leave-one-out CV (LOOCV)**.

6.3 Classification Metrics

For binary classification with predictions $\hat{y} \in \{0, 1\}$ and true labels $y \in \{0, 1\}$:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Definition 6.3.1: Precision, Recall, and F1

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Remark.

Accuracy is misleading on imbalanced datasets. A classifier that predicts “always negative” on a 99%-negative dataset gets 99% accuracy while detecting nothing. Use Precision, Recall, or F1.

The **ROC curve** plots True Positive Rate (Recall) vs. False Positive Rate $\frac{FP}{FP + TN}$ as the decision threshold varies. The **AUC** (area under the ROC curve) summarizes performance across all thresholds; AUC = 1 is perfect, AUC = 0.5 is random.

Part II

Supervised Learning

Chapter 7

Linear Regression

7.1 The Linear Model

Definition 7.1.1: Linear Regression

Given input $\mathbf{x} = (x_1, \dots, x_d)^T \in \mathbb{R}^d$, the linear model predicts a real-valued output via

$$f(\mathbf{x}; \mathbf{w}) = w_0 + \sum_{j=1}^d w_j x_j = \mathbf{w}^T \mathbf{x},$$

where the bias w_0 is absorbed by prepending a constant 1 to $\mathbf{x}$, so $\mathbf{w} \in \mathbb{R}^{d+1}$.

We stack n training examples into a design matrix $X \in \mathbb{R}^{n \times (d+1)}$ (rows are data points) and label vector $\mathbf{y} \in \mathbb{R}^n$. The predicted output vector is $\hat{\mathbf{y}} = X\mathbf{w}$.

7.2 Loss Functions

Definition 7.2.1: Mean Squared Error

$$L_{\text{MSE}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 = \frac{1}{n} \|X\mathbf{w} - \mathbf{y}\|^2.$$

MSE penalizes large errors quadratically and is differentiable everywhere. It is sensitive to outliers.

Definition 7.2.2: Mean Absolute Error

$$L_{\text{MAE}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n |y_i - \mathbf{w}^T \mathbf{x}_i|.$$

Robust to outliers but not differentiable at zero; requires subgradient methods.

Definition 7.2.3: Huber Loss

A smooth compromise between MSE and MAE, controlled by $\delta > 0$:

$$H_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta) & |r| > \delta. \end{cases}$$

Quadratic near zero (differentiable), linear in the tails (robust).

Remark.

The loss choice encodes a noise assumption. MSE corresponds to Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$; MAE to Laplacian noise; Huber to a robust mixture. For high-leverage points or outliers, MAE or Huber is preferred.

7.3 Ordinary Least Squares

Theorem 7.3.1: Normal Equations

The unique minimizer of L_{MSE} satisfies

$$X^T X \hat{\mathbf{w}} = X^T \mathbf{y},$$

and when X has full column rank, $\hat{\mathbf{w}} = (X^T X)^{-1} X^T \mathbf{y}$.

Proof

Expanding $L(\mathbf{w}) = \frac{1}{n}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ and differentiating:

$$\nabla_{\mathbf{w}} L = \frac{2}{n} X^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \implies X^T X \mathbf{w} = X^T \mathbf{y}.$$

The matrix $X^T X$ is positive semi-definite. Full column rank of X implies $X^T X \succ 0$, giving a unique solution via matrix inversion. $\square$

Fact 7.3.2

The level set $\{\mathbf{w} \mid (\mathbf{w} - \hat{\mathbf{w}})^T X^T X (\mathbf{w} - \hat{\mathbf{w}}) = C\}$ is an ellipsoid whose axis lengths scale with the eigenvalues of $(X^T X)^{-1}$. The gradient descent trajectory descends perpendicular to these contours.

Gradient descent iterates $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$, which expands to the per-weight update:

$$w_j \leftarrow w_j + \frac{2\eta}{n} \sum_{i=1}^n [y_i - f(\mathbf{x}_i; \mathbf{w})] x_{i,j}.$$

7.4 Probabilistic Interpretation

Assume $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon_i$ with $\epsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$. Then

$$P(y_i \mid \mathbf{x}_i; \mathbf{w}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \mathbf{w}^T \mathbf{x}_i)^2}{2\sigma^2}\right).$$

Proposition 7.4.1MLE $\equiv$ OLS under Gaussian Noise

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg \max_{\mathbf{w}} \sum_{i=1}^n \log P(y_i | \mathbf{x}_i; \mathbf{w}) = \arg \min_{\mathbf{w}} \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i)^2 = \hat{\mathbf{w}}_{\text{OLS}}.$$

The negative log-likelihood of i.i.d. Gaussian noise is exactly the MSE.

7.5 Collinearity

Definition 7.5.1: Collinearity

Two predictors x_j and x_k are **collinear** (or multicollinear in a group) if they are approximately linearly dependent. Perfectly collinear predictors make $X^T X$ singular.

Fact 7.5.2

When predictors are highly correlated, the MSE contours form a very elongated ellipsoid: the eigenvalues of $X^T X$ differ by orders of magnitude. The OLS estimate $\hat{\mathbf{w}} = (X^T X)^{-1} X^T \mathbf{y}$ is extremely sensitive to small perturbations in the data. Ridge regression (Chapter 9) resolves this by replacing $X^T X$ with $X^T X + \lambda I$, which is always invertible.

Example 7.5.1

Suppose $x_2 = 2x_1$ exactly. Then the 2×2 matrix $X^T X$ for (x_1, x_2) has a zero eigenvalue, and infinitely many $\hat{\mathbf{w}}$ achieve the same MSE. Any combination $w_1 + 2w_2 = \text{const}$ lies in the null space.

Chapter 8

Logistic Regression

8.1 From Regression to Classification

For binary classification $y \in \{0, 1\}$, we want to model $P(Y = 1 \mid X = \mathbf{x})$. Applying a linear model directly is problematic: outputs can fall outside $[0, 1]$. We instead compose the linear function with the **sigmoid**:

Definition 8.1.1: Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}, \quad z \in \mathbb{R}, \quad \sigma(z) \in (0, 1).$$

Properties: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; symmetric about 0; $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$.

The logistic regression model is:

$$p(\mathbf{x}) = P(Y = 1 \mid X = \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}.$$

The decision rule is $\hat{y} = \mathbf{1}[p(\mathbf{x}) > 0.5]$, equivalently $\hat{y} = \mathbf{1}[\mathbf{w}^T \mathbf{x} > 0]$. The **decision boundary** is the hyperplane $\mathbf{w}^T \mathbf{x} = 0$.

8.2 Log-Odds and the Logit Link

Proposition 8.2.1

Log-Odds Interpretation

Rearranging the logistic model gives the **logit** (log-odds) function:

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = \mathbf{w}^T \mathbf{x}.$$

Logistic regression is a *linear model for the log-odds*. Each weight w_j shifts the log-odds by w_j per unit increase in x_j .

8.3 Learning via Maximum Likelihood

Definition 8.3.1: Cross-Entropy Loss

Since $P(Y = y \mid X = \mathbf{x}) = p(\mathbf{x})^y [1 - p(\mathbf{x})]^{1-y}$, the negative log-likelihood over the training set is the **binary cross-entropy**:

$$L(\mathbf{w}) = - \sum_{i=1}^n \{y_i \log p(\mathbf{x}_i; \mathbf{w}) + (1 - y_i) \log(1 - p(\mathbf{x}_i; \mathbf{w}))\}.$$

Proposition 8.3.2

Convexity of Cross-Entropy

The cross-entropy loss is convex in $\mathbf{w}$, so gradient descent converges to the global minimum (if one exists). In contrast, using MSE on $p(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$ yields a non-convex objective with many local minima.

Proof

(Sketch) The Hessian of $L(\mathbf{w})$ equals $X^T D X$ where $D = \text{diag}(p_i(1 - p_i))$ is a diagonal matrix with non-negative entries. Hence the Hessian is positive semi-definite everywhere, confirming convexity. $\square$

8.4 Gradient and Gradient Descent

The gradient of the cross-entropy loss is remarkably clean:

$$\nabla_{\mathbf{w}} L = - \sum_{i=1}^n [y_i - p(\mathbf{x}_i; \mathbf{w})] \mathbf{x}_i.$$

The update rule for each weight w_j is:

$$w_j \leftarrow w_j - \eta \sum_{i=1}^n -[y_i - p(\mathbf{x}_i; \mathbf{w})] x_{i,j}.$$

Remark.

The gradient has the same form as in linear regression, with the prediction $p(\mathbf{x}_i; \mathbf{w}) = \sigma(\mathbf{w}^T \mathbf{x}_i)$ replacing $\mathbf{w}^T \mathbf{x}_i$. This structural similarity makes both models easy to unify in a single codebase.

Proposition 8.4.1

Well-Calibration

At the gradient optimum, setting $\nabla L = 0$ and examining the intercept term yields $\sum_i y_i = \sum_i p(\mathbf{x}_i; \hat{\mathbf{w}})$: the model's average predicted probability equals the empirical class frequency. Logistic regression is *well-calibrated* in this sense.

8.5 Multiclass: Softmax Regression

For K classes, we learn weight vectors $\mathbf{w}_1, \dots, \mathbf{w}_K$ and compute class logits $z_k = w_{k0} + \mathbf{w}_k^T \mathbf{x}$. The **softmax** function converts logits to a probability distribution:

Definition 8.5.1: Softmax

$$P(Y = k \mid X = \mathbf{x}) = \frac{\exp(z_k)}{\sum_{k'=1}^K \exp(z_{k'})}, \quad k = 1, \dots, K.$$

The outputs are non-negative and sum to 1. As $\alpha \rightarrow \infty$, $\text{softmax}(\alpha z)$ concentrates all mass on $\arg \max_k z_k$.

Fact 8.5.2

Softmax generalizes sigmoid: for $K = 2$, $P(Y = 1) = \exp(z_1)/(\exp(z_1) + \exp(z_2)) = \sigma(z_1 - z_2)$.

The loss is the multiclass cross-entropy:

$$L = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log \hat{y}_{ik},$$

where $\mathbf{y}_i$ is a one-hot vector and $\hat{y}_{ik} = P(Y = k \mid \mathbf{x}_i)$.

8.6 Limitations

Logistic regression produces a *linear* decision boundary $\mathbf{w}^T \mathbf{x} = 0$. For problems like XOR (where no hyperplane separates the classes), it fails. The natural remedy is to either: (a) engineer nonlinear features $\phi(\mathbf{x})$, or (b) cascade multiple logistic regression models — which gives a multilayer neural network (Part IV).

Chapter 9

Regularization

9.1 The Overfitting Problem

Without constraints, empirical risk minimization can fit training data perfectly while failing to generalize. The bias-variance decomposition (Chapter 1) shows that model complexity is the lever: increasing it reduces bias but raises variance. Regularization adds an explicit complexity penalty to the loss.

9.2 Ridge Regression (ℓ_2 Regularization)

Definition 9.2.1: Ridge Regression

Ridge regression augments the MSE objective with an ℓ_2 penalty on the weights:

$$L_{\text{ridge}}(\mathbf{w}) = \frac{1}{n} \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2, \quad \lambda > 0.$$

The bias term w_0 is conventionally excluded from penalization.

Theorem 9.2.2: Ridge Closed Form

The unique minimizer is

$$\hat{\mathbf{w}}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}.$$

The matrix $X^T X + \lambda I$ is positive definite for any $\lambda > 0$, resolving collinearity.

Proof

$\nabla L_{\text{ridge}} = \frac{2}{n} X^T (X\mathbf{w} - \mathbf{y}) + 2\lambda\mathbf{w} = 0$ gives $(X^T X + n\lambda I)\mathbf{w} = X^T \mathbf{y}$, and absorbing the factor of n into λ . $\square$

Proposition 9.2.3

Bayesian Interpretation of Ridge

Ridge is equivalent to MAP estimation with a Gaussian prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \frac{1}{2\lambda} I)$ on the weights. The ℓ_2 penalty is the negative log-prior.

Remark.

Geometrically, the Ridge constraint $\|\mathbf{w}\|^2 \leq t$ defines a sphere. The solution is where the OLS contours first touch the sphere. All weights are shrunk toward zero, but none are exactly zero.

9.3 Lasso (ℓ_1 Regularization)

Definition 9.3.1: Lasso

$$L_{\text{lasso}}(\mathbf{w}) = \frac{1}{n} \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1, \quad \|\mathbf{w}\|_1 = \sum_{j=1}^d |w_j|.$$

Proposition 9.3.2

Sparsity of Lasso

The ℓ_1 constraint region is a cross-polytope (diamond in 2D). Its corners lie on coordinate axes, so the OLS contours frequently touch the constraint boundary at a corner — setting some weights to *exactly zero*. Lasso performs automatic feature selection.

Unlike Ridge, Lasso has no closed-form solution (the ℓ_1 norm is non-differentiable at 0). It is solved via coordinate descent or proximal gradient methods.

Proposition 9.3.3

Bayesian Interpretation of Lasso

Lasso is equivalent to MAP estimation with a Laplace prior $p(w_j) \propto \exp(-\lambda|w_j|)$. The heavier tails of the Laplace distribution encourage sparsity.

9.4 Elastic Net

Definition 9.4.1: Elastic Net

Elastic Net combines ℓ_1 and ℓ_2 penalties:

$$L_{\text{EN}}(\mathbf{w}) = \frac{1}{n} \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda_1 \|\mathbf{w}\|_1 + \lambda_2 \|\mathbf{w}\|^2.$$

It inherits sparsity from Lasso and the grouping effect (correlated features get similar weights) from Ridge.

9.5 Regularization for Logistic Regression

The same penalties apply to classification. The regularized cross-entropy objectives are:

$$L_{L2}(\mathbf{w}) = - \sum_{i=1}^n \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\} + \lambda \sum_{j=1}^d w_j^2,$$

$$L_{L1}(\mathbf{w}) = - \sum_{i=1}^n \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\} + \lambda \sum_{j=1}^d |w_j|.$$

ℓ_2 regularization prevents the weights from growing arbitrarily large when classes are linearly separable (which would push logistic outputs to 0/1 and cause numerical issues).

9.6 Effect on Bias and Variance

Proposition 9.6.1

Bias-Variance Tradeoff in Ridge

As $\lambda \rightarrow 0$, $\hat{\mathbf{w}}_{\text{ridge}} \rightarrow \hat{\mathbf{w}}_{\text{OLS}}$ (zero bias, full variance). As $\lambda \rightarrow \infty$, $\hat{\mathbf{w}}_{\text{ridge}} \rightarrow \mathbf{0}$ (high bias, zero variance). The optimal λ is chosen by cross-validation.

Chapter 10

Support Vector Machines

10.1 Hard-Margin SVM

For binary classification with labels $y_i \in \{+1, -1\}$ and a linearly separable dataset, we seek the hyperplane $\mathbf{w}^T \mathbf{x} + w_0 = 0$ with *maximum margin*.

Definition 10.1.1: Margin

The **functional margin** of point $(\mathbf{x}_i, y_i)$ is $y_i(\mathbf{w}^T \mathbf{x}_i + w_0)$. The **geometric margin** (distance from the hyperplane) is $y_i(\mathbf{w}^T \mathbf{x}_i + w_0) / \|\mathbf{w}\|$. The margin of the classifier is $\min_i y_i(\mathbf{w}^T \mathbf{x}_i + w_0) / \|\mathbf{w}\|$.

By rescaling $\mathbf{w}$ so that $\min_i y_i(\mathbf{w}^T \mathbf{x}_i + w_0) = 1$, the margin becomes $1 / \|\mathbf{w}\|$. Maximizing the margin is equivalent to:

Definition 10.1.2: Hard-Margin SVM (Primal)

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1 \quad \forall i.$$

The points achieving equality, $y_i(\mathbf{w}^T \mathbf{x}_i + w_0) = 1$, are the **support vectors**.

10.2 Dual Formulation

Introducing Lagrange multipliers $\alpha_i \geq 0$ for each constraint gives the dual problem:

Definition 10.2.1: SVM Dual

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0.$$

Proposition 10.2.2

KKT Complementary Slackness

At optimality: $\alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + w_0) - 1] = 0$. Thus $\alpha_i > 0$ only for support vectors. The weight vector $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$ is a linear combination of support vectors alone.

Prediction uses only inner products with support vectors:

$$f(\mathbf{x}) = w_0 + \sum_{(\mathbf{x}', y') \in \text{SV}} y' \alpha' \langle \mathbf{x}', \mathbf{x} \rangle.$$

10.3 Soft-Margin SVM and Hinge Loss

When the data is not linearly separable, we allow margin violations via **slack variables** $\xi_i \geq 0$:

Definition 10.3.1: Soft-Margin SVM

$$\min_{\mathbf{w}, w_0, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

$C > 0$ trades margin width against training error. Substituting $\xi_i = \max(1 - y_i(\mathbf{w}^T \mathbf{x}_i + w_0), 0)$ gives the unconstrained objective:

Proposition 10.3.2

SVM as Regularized Hinge Loss

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(1 - y_i(w_0 + \mathbf{w}^T \mathbf{x}_i), 0).$$

The **hinge loss** $\ell_{\text{hinge}}(y, f) = \max(1 - yf, 0)$ is convex and zero when the point is correctly classified with margin ≥ 1 .

10.4 The Kernel Trick

The dual SVM objective depends on the data only through inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$. If we apply a feature map $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ (possibly $m = \infty$), we can replace inner products with a **kernel function** $k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$, *without ever computing $\phi(\mathbf{x})$ explicitly*.

Definition 10.4.1: Common Kernels

Kernel	Formula	Remark
Linear	$k(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T \mathbf{v}$	Standard SVM
Polynomial	$k(\mathbf{u}, \mathbf{v}) = (1 + \mathbf{u}^T \mathbf{v})^D$	Degree- D polynomial map
RBF / Gaussian	$k(\mathbf{u}, \mathbf{v}) = \exp(-\gamma \ \mathbf{u} - \mathbf{v}\ ^2)$	Infinite-dimensional ϕ

Proposition 10.4.2

Mercer's Condition

A symmetric function k is a valid kernel (i.e., corresponds to some ϕ) if and only if the Gram matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is positive semi-definite for all data sets.

With a kernel, prediction becomes:

$$f(\mathbf{x}) = w_0 + \sum_{(\mathbf{x}', y') \in \text{SV}} \alpha' y' k(\mathbf{x}', \mathbf{x}).$$

The RBF kernel is equivalent to an infinite Taylor expansion: $k(\mathbf{x}_i, \mathbf{x}_j) = C \sum_{n=0}^{\infty} \frac{\langle \mathbf{x}_i, \mathbf{x}_j \rangle^n}{n!}$, giving it universal approximation capability.

Chapter 11

K -Nearest Neighbors

11.1 The Algorithm

Definition 11.1.1: K -Nearest Neighbors (KNN)

Given a test point $\mathbf{x}$, find the K training points $\mathcal{N}_K(\mathbf{x}) = \{\mathbf{x}_{(1)}, \dots, \mathbf{x}_{(K)}\}$ closest under a distance metric $d(\cdot, \cdot)$.

- **Classification:** $\hat{y} = \text{mode}\{y_i : \mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})\}$ (majority vote).
- **Regression:** $\hat{y} = \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x})} y_i$ (average).

KNN is a **lazy learner**: there is no training phase; all computation occurs at prediction time.

11.2 Distance Metrics

The choice of distance metric critically affects KNN performance.

Definition 11.2.1: Minkowski Distance

For $p \geq 1$:

$$d_p(\mathbf{u}, \mathbf{v}) = \left(\sum_{j=1}^d |u_j - v_j|^p \right)^{1/p}.$$

- $p = 2$: Euclidean distance (most common).
- $p = 1$: Manhattan (taxicab) distance. More robust to outliers.
- $p \rightarrow \infty$: Chebyshev distance $\max_j |u_j - v_j|$.

Remark.

Features should be normalized before applying KNN. If one feature has range $[0, 1000]$ and another $[0, 1]$, the first will dominate the distance computation.

11.3 Choosing K and the Bias-Variance Tradeoff

Proposition 11.3.1

Effect of K

Small K (e.g., $K = 1$): the decision boundary follows every training point, yielding low bias but high variance (overfitting). Large K : smoother boundary, higher bias, lower variance. The optimal K is selected by cross-validation.

Proposition 11.3.2

Asymptotic Optimality

As $n \rightarrow \infty$ with $K \rightarrow \infty$ and $K/n \rightarrow 0$, the 1-NN error rate converges to at most $2R^*$ where R^* is the Bayes error rate. For $K \geq 1$, KNN is universally consistent.

11.4 Curse of Dimensionality

Fact 11.4.1

In d dimensions, to capture a fraction f of the data volume in a neighborhood, the side length of the neighborhood must be $f^{1/d}$. For $f = 0.01$ and $d = 10$, this is $0.01^{0.1} \approx 0.63$ — nearly the full feature range. Nearest neighbors cease to be meaningfully “near” in high dimensions.

Proposition 11.4.2

Computational Complexity

Naïve KNN requires $O(nd)$ distance computations per query. With n training points and q test points, total cost is $O(qnd)$. Data structures such as k -d trees reduce average-case query time to $O(d \log n)$ for low d , but lose their advantage as d grows.

Chapter 12

Naïve Bayes

12.1 Generative vs. Discriminative Classifiers

Definition 12.1.1: Discriminative Model

Directly models the posterior $P(Y | X)$. Examples: logistic regression, SVM.

Definition 12.1.2: Generative Model

Models the joint distribution $P(X, Y) = P(X | Y)P(Y)$, then applies Bayes' rule to obtain $P(Y | X)$.

12.2 Bayes' Rule for Classification

Definition 12.2.1: Bayes Classifier

Given a new point $\mathbf{x}$, predict:

$$\hat{y} = \arg \max_y P(Y = y | X = \mathbf{x}) = \arg \max_y \frac{P(X = \mathbf{x} | Y = y) P(Y = y)}{P(X = \mathbf{x})}.$$

Since $P(X = \mathbf{x})$ is constant across classes:

$$\hat{y} = \arg \max_y P(X = \mathbf{x} | Y = y) P(Y = y).$$

$P(Y = y)$ is the **prior** (estimated from class frequencies in training data); $P(X | Y = y)$ is the **class-conditional likelihood**.

Fact 12.2.2

The Bayes classifier minimizes the probability of misclassification given perfect knowledge of $P(Y | X)$. The resulting error is the **Bayes error rate** $R^* = \mathbb{E}[1 - \max_y P(Y = y | X)]$, a fundamental lower bound on classification error.

12.3 The Naïve Bayes Assumption

The class-conditional $P(X | Y = y)$ is a d -dimensional distribution. Estimating it directly requires exponentially many parameters (e.g., a table with 2^d entries for binary features). The key simplification:

Definition 12.3.1: Naïve Bayes (Conditional Independence) Assumption

Features are assumed conditionally independent given the class label:

$$P(X_1, \dots, X_d | Y = y) = \prod_{j=1}^d P(X_j | Y = y).$$

This reduces estimation to d separate 1-D problems.

The full prediction rule becomes:

$$\hat{y} = \arg \max_y P(Y = y) \prod_{j=1}^d P(X_j = x_j | Y = y).$$

Remark.

The “naïve” assumption is almost always violated in practice — (“likes cats” and “likes dogs” are surely correlated). Yet Naïve Bayes remains surprisingly effective, especially for text classification, because the decision boundary is more robust than the probabilities themselves.

12.4 Parameter Estimation

Categorical features: Estimate $P(X_j = v | Y = y) = \frac{\#\{i: x_{ij}=v, y_i=y\}}{\#\{i: y_i=y\}}$ by counting.

Gaussian Naïve Bayes: Assume $P(X_j | Y = y) = \mathcal{N}(\mu_{jy}, \sigma_{jy}^2)$ for continuous features. Estimate μ_{jy} and σ_{jy}^2 by class-conditional sample mean and variance.

12.5 Laplace Smoothing

Definition 12.5.1: Laplace Smoothing

If a feature value never appears in class y , the estimate $P(X_j = v | Y = y) = 0$ zeros out the entire product. Laplace smoothing adds a pseudocount $\alpha > 0$:

$$P(X_j = v | Y = y) = \frac{\#\{i: x_{ij} = v, y_i = y\} + \alpha}{\#\{i: y_i = y\} + \alpha |\mathcal{V}_j|},$$

where $|\mathcal{V}_j|$ is the number of distinct values of feature j . For $\alpha = 1$ this is called **add-one smoothing**.

In practice, computations are done in log-space to avoid numerical underflow from multiplying many probabilities:

$$\hat{y} = \arg \max_y \left[\log P(Y = y) + \sum_{j=1}^d \log P(X_j = x_j | Y = y) \right].$$

Chapter 13

Decision Trees and Ensembles

13.1 Decision Trees

Definition 13.1.1: Decision Tree

A decision tree is a hierarchical model that recursively partitions the feature space using axis-aligned binary splits. Each internal node tests a feature x_j against a threshold t ; each leaf stores a predicted value (majority class or mean target).

A tree with M leaves partitions $\mathbb{R}^d$ into M rectangular regions $R_1, \dots, R_M$. Predictions are constant within each region:

$$f(\mathbf{x}) = \sum_{m=1}^M c_m \mathbf{1}[\mathbf{x} \in R_m].$$

13.2 Splitting Criteria

Greedy tree learning chooses the split (j, t) that maximizes the decrease in impurity.

Definition 13.2.1: Entropy and Information Gain

For a set S with class proportions $p_1, \dots, p_K$, the **Shannon entropy** is:

$$H(S) = - \sum_{k=1}^K p_k \log_2 p_k.$$

The **information gain** of splitting S on feature j at threshold t is:

$$\text{IG}(S, j, t) = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R),$$

where S_L, S_R are the left and right subsets after the split. The ID3/C4.5 algorithm greedily maximizes information gain.

Definition 13.2.2: Gini Impurity

$$G(S) = 1 - \sum_{k=1}^K p_k^2 = \sum_{k \neq k'} p_k p_{k'}.$$

$G(S) = 0$ for a pure node (one class); $G(S) = 1 - 1/K$ for maximum impurity. CART uses Gini impurity.

Proposition 13.2.3

For regression trees, the splitting criterion minimizes the weighted sum of within-region MSE:

$$\frac{1}{|S_L|} \sum_{\mathbf{x}_i \in S_L} (y_i - \bar{y}_{S_L})^2 + \frac{1}{|S_R|} \sum_{\mathbf{x}_i \in S_R} (y_i - \bar{y}_{S_R})^2.$$

13.3 Overfitting and Pruning

A fully-grown tree (one leaf per training point) memorizes training data. **Cost-complexity pruning** replaces subtrees with leaves while the gain is below a threshold. Alternatively, restrict tree depth, minimum leaf size, or minimum impurity decrease.

13.4 Random Forests

Definition 13.4.1: Bagging (Bootstrap Aggregating)

Train B trees on independently bootstrapped datasets $\mathcal{D}_1^*, \dots, \mathcal{D}_B^*$ (each drawn with replacement from $\mathcal{D}$). Average their predictions (regression) or take majority vote (classification):

$$f_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}).$$

Bagging reduces variance without increasing bias.

Definition 13.4.2: Random Forest

Extend bagging by also randomly subsampling $m \approx \sqrt{d}$ features at each split. This decorrelates the trees, further reducing variance. Random forests are robust, parallelizable, and provide out-of-bag error estimates.

13.5 Boosting

Boosting constructs an additive model $f(x) = \sum_{k=1}^K \alpha_k g(x; \gamma_k)$ by **sequentially** fitting weak learners g (typically shallow trees), each one correcting the errors of its predecessors.

Definition 13.5.1: Forward Stagewise Additive Modeling

Initialize $f_0(x) = 0$. At each step $k = 1, \dots, K$:

1. Fit: $(\alpha_k, \gamma_k) = \arg \min_{\alpha, \gamma} \sum_{i=1}^n L(y_i, f_{k-1}(x_i) + \alpha g(x_i; \gamma))$.
2. Update: $f_k(x) = f_{k-1}(x) + \alpha_k g(x; \gamma_k)$.

Definition 13.5.2: AdaBoost

For binary classification $y_i \in \{+1, -1\}$ with exponential loss $L(y, f) = \exp(-yf)$, the forward stagewise procedure reduces to AdaBoost:

1. Initialize sample weights $u_i^{(1)} = 1$.
2. For $k = 1, \dots, K$: train g_k on reweighted data, compute weighted error $\varepsilon_k = \frac{\sum_i u_i^{(k)} \mathbf{1}[y_i \neq g_k(x_i)]}{\sum_i u_i^{(k)}}$, set $\alpha_k = \frac{1}{2} \log \frac{1-\varepsilon_k}{\varepsilon_k}$, then update $u_i^{(k+1)} = u_i^{(k)} \exp(-\alpha_k y_i g_k(x_i))$.
3. Output $f(x) = \text{sign}(\sum_k \alpha_k g_k(x))$.

Misclassified points receive higher weight in the next iteration; correctly classified points receive lower weight.

Proposition 13.5.3

Under exponential loss, the stagewise procedure exactly corresponds to fitting each weak learner g_k on the reweighted training set, with weights $u_i^{(k)} = \exp(-y_i f_{k-1}(x_i))$.

Definition 13.5.4: Gradient Boosting

For a general differentiable loss L , at step k compute the **pseudo-residuals**

$$r_i = - \left. \frac{\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}}{f = f_{k-1}} \right|, \quad ,$$

and fit g_k to the pseudo-residuals. For squared error loss, this is exactly fitting the residuals $r_i = y_i - f_{k-1}(x_i)$.

Remark.

XGBoost and LightGBM are highly optimized gradient boosting implementations that dominate tabular data competitions. They add second-order Taylor expansions of the loss, regularization on leaf weights, and efficient histogram-based split finding.

Part III

Unsupervised Learning

Chapter 14

Clustering

14.1 The Unsupervised Setting

In supervised learning we are given labeled pairs (x_i, y_i) . In the *unsupervised* setting we observe only $\{x_1, \dots, x_n\}$ and must discover structure without any target signal. Clustering is the task of grouping examples so that points within the same cluster are similar and points in different clusters are dissimilar.

Definition 14.1.1: Clustering

Given data $\{x_1, \dots, x_n\} \subset \mathbb{R}^d$ and a number of clusters k , find a partition $\{S_1, \dots, S_k\}$ of the data and representative centroids $\{c_1, \dots, c_k\}$ such that the **intra-cluster distortion** is minimised:

$$\min_{\{S_j\}, \{c_j\}} \sum_{j=1}^k \sum_{x \in S_j} \|x - c_j\|^2.$$

14.2 K-Means Algorithm

K-means (Steinhaus 1950 / Lloyd 1957) is an iterative algorithm that alternates between two steps.

Fact 14.2.1

K-Means Algorithm. Input: $\{x_1, \dots, x_n\}$, k .

Initialise: choose $c_1, \dots, c_k$ (e.g. random sample or K-means++ — see below).

Repeat until centroids do not change:

1. *Assignment step.* For every x_i set $j^* = \arg \min_{j=1:k} \|x_i - c_j\|^2$ and assign $S_{j^*} \leftarrow S_{j^*} \cup \{x_i\}$.
2. *Update step.* For $j = 1, \dots, k$ recompute $c_j \leftarrow \frac{1}{|S_j|} \sum_{x \in S_j} x$.

An equivalent continuous objective reformulates the problem as:

$$\min_{c_1, \dots, c_k} f(c_1, \dots, c_k) = \frac{1}{2} \sum_{i=1}^n \min_{j=1:k} \|x_i - c_j\|^2.$$

The gradient update on centroid c_j with learning rate η gives the SGD K-means rule:

$$c_j \leftarrow (1 - \eta) c_j + \eta x_i \quad (\text{for } x_i \text{ whose closest centroid is } c_j).$$

14.3 Convergence

Theorem 14.3.1: K-Means Convergence

K-means always terminates. At every iteration the objective $\sum_j \sum_{x \in S_j} \|x - c_j\|^2$ is non-increasing:

- The assignment step only moves each x to a closer (or equally close) centroid.
- The update step sets c_j to the (unique) minimiser of $\sum_{x \in S_j} \|x - c_j\|^2$, namely the mean.

Since there are finitely many possible partitions, the algorithm must stop.

K-means converges to a *local* optimum; the global optimum is NP-hard in general. Running the algorithm multiple times with different initialisations mitigates this.

14.4 Initialisation: K-Means++

Definition 14.4.1: K-Means++ Initialisation

Let $d_j(x)$ be the distance from x to the closest of the j centroids already chosen.

1. Choose c_1 uniformly at random from $\{x_1, \dots, x_n\}$.
2. For $j = 2, \dots, k$: choose c_j with probability proportional to $d_{j-1}(x)^2$, i.e.

$$P(x = c_j) = \frac{d_{j-1}(x)^2}{\sum_{i=1}^n d_{j-1}(x_i)^2}.$$

This ensures each new centroid is likely to be far from all previous ones, providing better coverage of the data.

14.5 Connection to Gaussian Mixture Models

Consider a uniform mixture of k spherical Gaussians with means at the K-means centroids:

$$P(x) \propto \frac{1}{k} \sum_{j=1}^k \exp(-\|x - c_j\|^2).$$

Because of exponential decay, only the closest centroid $z(x) = \arg \min_{c_j} \|x - c_j\|^2$ contributes appreciably:

$$P(X) \approx \text{const} \times \exp\left(-\sum_{i=1}^n \|x_i - z(x_i)\|^2\right).$$

Hence maximising the likelihood under this model is equivalent to minimising the K-means objective — K-means performs *hard* EM on a Gaussian mixture (vs. soft EM which maintains probabilistic assignments).

14.6 Limitations of K-Means

Remark.

- K-means assumes **spherical clusters** of similar size and density. It fails on elongated, non-convex, or heterogeneous clusters.
- The number of clusters k must be specified in advance. A common heuristic is the *elbow method*: plot the objective vs. k and pick the “knee” of the curve.
- Because the partitioning space is discrete but large, a greedy algorithm may find a poor local minimum;

running with multiple random restarts or K-means++ is recommended.

Chapter 15

Dimensionality Reduction

15.1 Motivation

High-dimensional data often lies near a lower-dimensional manifold. Dimensionality reduction finds a mapping $x \in \mathbb{R}^d \rightarrow z \in \mathbb{R}^q$ with $q \ll d$ that preserves as much information as possible. Uses include visualisation, removing redundant/correlated features, and reducing computational cost.

15.2 Principal Component Analysis

Definition 15.2.1: PCA

Principal Component Analysis (PCA) finds the q orthonormal directions (principal components) in $\mathbb{R}^d$ that capture the most variance in the data. Equivalently, the projection onto these q directions minimises the mean reconstruction error.

The two interpretations are equivalent:

Maximum variance	maximise variance of projections
Minimum reconstruction error	minimise $\sum_i \ x_i - \hat{x}_i\ ^2$

Procedure (Variance View)

1. **Centre the data:** $X \leftarrow X - \bar{X}$ where $\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i$.
2. Find the q eigenvectors of $X^\top X$ with the q largest eigenvalues; call them $v_1, \dots, v_q$.
3. Project: $z_i = [v_1^\top x_i, \dots, v_q^\top x_i]^\top \in \mathbb{R}^q$.

Derivation (First Principal Component)

For a unit vector v ($\|v\|_2 = 1$) the variance of the projections Xv is (after centring, $\bar{X}v = 0$):

$$\text{Var}(Xv) = \sum_i (v^\top x_i)^2 = \|Xv\|_2^2 = v^\top X^\top X v.$$

We want:

$$\max_{v: \|v\|_2=1} v^\top X^\top X v.$$

Introducing a Lagrange multiplier λ and setting the gradient to zero:

$$\nabla[v^\top X^\top X v - \lambda v^\top v] = 2(X^\top X - \lambda I)v = 0 \implies \boxed{X^\top X v = \lambda v.}$$

The maximum is attained at the eigenvector with the *largest* eigenvalue. The q principal components are the top- q eigenvectors.

Proposition 15.2.2

Variance Explained

If we keep the first q principal components with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$, the fraction of total variance retained is

$$\frac{\sum_{k=1}^q \lambda_k}{\sum_{k=1}^p \lambda_k}.$$

Plot this vs. q (a *scree plot*) to choose q .

15.3 PCA via Singular Value Decomposition

Definition 15.3.1: SVD

Every matrix $M \in \mathbb{R}^{n \times d}$ admits a factorisation

$$M = U \Sigma V^\top$$

where $U \in \mathbb{R}^{n \times n}$ (columns = eigenvectors of $MM^\top$), $\Sigma \in \mathbb{R}^{n \times d}$ (diagonal, singular values $\sigma_i = \sqrt{\lambda_i}$), and $V \in \mathbb{R}^{d \times d}$ (columns = eigenvectors of $M^\top M$).

For PCA with centred data matrix X : compute $X = U \Sigma V^\top$; the columns of V are the principal directions, and $XV = U \Sigma$ are the principal component scores.

15.4 Autoencoders: Non-linear PCA

A linear autoencoder with tied weights (W and $W^\top$) trained to minimise reconstruction error $\sum_{x \in D} \|x - \hat{x}\|^2$ recovers the PCA subspace. A **deep autoencoder** replaces the linear layers with non-linear networks (encoder f_θ and decoder g_ϕ):

$$z = f_\theta(x) \in \mathbb{R}^k, \quad \hat{x} = g_\phi(z) \in \mathbb{R}^d, \quad \min_{\theta, \phi} \sum_{x \in D} \|x - g_\phi(f_\theta(x))\|^2.$$

The encoder maps $x \in \mathbb{R}^d$ to a low-dimensional code $z \in \mathbb{R}^k$ ($k \ll d$); the decoder reconstructs $\hat{x}$. Deep autoencoders can capture non-linear manifold structure that PCA misses.

Remark.

Autoencoder as pre-training. When labelled data is scarce, one can (1) train an autoencoder on the large unlabelled corpus to learn a rich latent representation, then (2) attach a classifier head to the encoder and fine-tune on the small labelled set. This is an early form of *self-supervised* pre-training.

15.5 Limitations of PCA

- PCA is a **linear** projection; it fails when data lies on a non-linear manifold.
- It assumes approximately Gaussian-distributed features.
- Sensitive to scale; features should be standardised before applying PCA.

Chapter 16

Learning Paradigms

16.1 A Taxonomy of Learning

Machine learning methods differ primarily in what supervision signal is available during training.

Definition 16.1.1: Supervised Learning

Training data consists of labeled pairs $\{(x_i, y_i)\}_{i=1}^n$. The goal is to learn a mapping $f : \mathcal{X} \rightarrow \mathcal{Y}$ that generalises to unseen inputs. Examples: linear/logistic regression, SVMs, neural networks, decision trees.

Definition 16.1.2: Unsupervised Learning

Only unlabeled examples $\{x_i\}_{i=1}^n$ are available. Goals include discovering clusters, learning compact representations, estimating densities, and generating new samples. Examples: K-means, PCA, autoencoders, GMMs, GANs.

Definition 16.1.3: Semi-Supervised Learning

A small labelled set $\mathcal{L} = \{(x_i, y_i)\}$ is supplemented by a large unlabelled set $\mathcal{U} = \{x_j\}$. The unlabelled data can reveal the structure of $P(X)$, which (under the manifold or cluster assumptions) helps improve decision boundaries learned from $\mathcal{L}$.

Definition 16.1.4: Self-Supervised Learning

Labels are derived automatically from the data itself. A *pretext task* (e.g. predict a masked token, predict the next token, predict image rotation) provides a supervised training signal without human annotation. The learned representations are then transferred to downstream tasks. Modern large language models (GPT, BERT) are trained this way.

Definition 16.1.5: Few-Shot Learning

At test time the model must generalise to new classes or tasks with only a handful of labelled examples (the *support set*). Meta-learning approaches (e.g. MAML, Prototypical Networks) aim to “learn how to learn” from prior tasks so adaptation to a new task requires minimal data.

16.2 Transfer Learning

In **transfer learning** a model pre-trained on a large source task is adapted to a target task.

- **Feature extraction:** freeze pre-trained weights; train only a new head on the target task.
- **Fine-tuning:** initialise from pre-trained weights; continue training all (or some) layers on the target task with a small learning rate.

Transfer learning is especially powerful when source and target distributions overlap (e.g. ImageNet $\rightarrow$ medical imaging; GPT pre-training $\rightarrow$ task-specific RLHF fine-tuning).

16.3 Reinforcement Learning

In **reinforcement learning** (RL) an agent interacts with an environment: at each step it observes a state s_t , takes an action a_t , receives a reward r_t , and transitions to s_{t+1} . The goal is to learn a *policy* $\pi(a|s)$ that maximises expected cumulative reward $\mathbb{E}[\sum_t \gamma^t r_t]$, where $\gamma \in [0, 1)$ is a discount factor. RL is covered in depth in Part VIII.

Part IV

Deep Learning

Chapter 17

Neural Network Foundations

17.1 The Single Neuron

The basic computational unit is an artificial neuron. Given input $x \in \mathbb{R}^d$, it computes

$$f(x) = g(w^\top x + b),$$

where $w \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ is the bias, and $g : \mathbb{R} \rightarrow \mathbb{R}$ is a non-linear *activation function*. For the squared-loss training objective $L = \frac{1}{2}(y - w^\top x - b)^2$, the gradient with respect to w is $\nabla_w L = -(y - w^\top x - b)x$.

17.2 Multi-Layer Perceptrons

Definition 17.2.1: Feedforward Neural Network

A feedforward network with L layers is defined by the forward pass:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g^{[l]}(z^{[l]}), \quad l = 1, \dots, L,$$

with $a^{[0]} = x$. The network output is $\hat{y} = a^{[L]}$.

Stacking the layers, the full network computes

$$f(x) = g^{[L]}(W^{[L]} g^{[L-1]}(\dots g^{[1]}(W^{[1]}x + b^{[1]}) \dots) + b^{[L]}).$$

Proposition 17.2.2

Necessity of Non-Linearity

If every $g^{[l]}$ is the identity, then $f(x) = (W^{[L]} \dots W^{[1]})x + b'$, a single affine map. Composing linear layers produces one linear layer; depth buys expressiveness only with non-linearities.

17.3 Activation Functions

Name	Formula $g(z)$	$g'(z)$	Notes
Sigmoid	$1/(1 + e^{-z})$	$g(1 - g)$	saturates; not zero-centred
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$1 - g^2$	zero-centred; still saturates
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$	fast; <i>dying neuron</i> risk
Leaky ReLU	$\max(\varepsilon z, z)$	ε or 1	fixes dead neurons

Remark.

Vanishing gradients. With sigmoid, $|\sigma'(z)| \leq 0.25$, so each layer multiplies the gradient by at most 0.25. For an L -layer network the gradient magnitude decays as $(0.25)^L$. ReLU avoids saturation on the positive half-line and is the **default choice for hidden layers**.

Definition 17.3.1: Output Layer Design

Choose the final activation and loss to match the task:

- Binary classification: sigmoid output, $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
- Multi-class (C classes): softmax $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$, cross-entropy loss.
- Regression: linear output (no activation), squared loss.

Fact 17.3.2

The *Universal Approximation Theorem* (Cybenko 1989) states that a single hidden layer of sufficient width with a non-polynomial activation can approximate any continuous function on a compact domain to arbitrary accuracy. Deeper networks achieve the same approximation with exponentially fewer neurons.

Chapter 18

Training Deep Networks

18.1 Backpropagation

Training minimises a loss L by gradient descent. Gradients are computed via the chain rule in a single backward pass through the computation graph.

Definition 18.1.1: Backpropagation (Matrix Form)

Given the forward pass $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$, $a^{[l]} = g^{[l]}(z^{[l]})$, the backward pass from $l = L$ down to 1:

$$\begin{aligned}\nabla_{z^{[l]}} L &= \nabla_{a^{[l]}} L \odot (g^{[l]})'(z^{[l]}), \\ \nabla_{W^{[l]}} L &= \nabla_{z^{[l]}} L \cdot (a^{[l-1]})^\top, \\ \nabla_{a^{[l-1]}} L &= (W^{[l]})^\top \nabla_{z^{[l]}} L.\end{aligned}$$

Modern frameworks (PyTorch, JAX) implement this via *automatic differentiation*: they record a computation graph during the forward pass and apply the chain rule in reverse automatically.

18.2 Weight Initialisation

Definition 18.2.1: Initialisation Schemes

- **Xavier (Glorot)**: $W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$. Designed for tanh/sigmoid to preserve gradient variance.
- **He**: $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$. Designed for ReLU.

18.3 Batch Normalisation

Definition 18.3.1: Batch Normalisation (Ioffe & Szegedy 2015)

For mini-batch $\{x^{(i)}\}_{i=1}^m$ and feature k , let $\mu_k = \frac{1}{m} \sum_i x_k^{(i)}$, $\sigma_k^2 = \frac{1}{m} \sum_i (x_k^{(i)} - \mu_k)^2$. Batch norm:

$$\hat{x}_k^{(i)} = \frac{x_k^{(i)} - \mu_k}{\sqrt{\sigma_k^2 + \varepsilon}}, \quad y_k^{(i)} = \gamma_k \hat{x}_k^{(i)} + \beta_k,$$

where γ_k, β_k are learned per-feature scale and shift.

Batch norm keeps activation distributions stable across layers, allowing higher learning rates and faster convergence. It also acts as a regulariser.

18.4 Dropout

Definition 18.4.1: Dropout (Srivastava et al. 2014)

During training, independently zero each neuron with probability p ; scale surviving activations by $\frac{1}{1-p}$ (*inverted dropout*). At inference, use all neurons without scaling.

Dropout prevents co-adaptation of neurons and acts as implicit ensemble averaging, reducing overfitting. Typical values: $p = 0.2$ – 0.5 .

18.5 Optimisation

SGD with Momentum:

$$v \leftarrow \beta v - \eta \nabla_{\theta} L, \quad \theta \leftarrow \theta + v.$$

Definition 18.5.1: Adam (Kingma & Ba 2015)

Maintains exponential moving averages of gradients m_t and squared gradients v_t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Bias-corrected estimates $\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$. Update:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}.$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

18.6 Learning Rate Schedules

The non-convex DNN loss surface benefits from a large η early (escape saddle points) and small η later (fine convergence).

- **Step decay:** multiply η by $\gamma < 1$ every k epochs.
- **Cosine annealing:** $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$.
- **Warmup:** linearly increase η for the first few steps to stabilise early training (used in Transformers).

18.7 Data Augmentation and Mixup

Data augmentation enlarges the effective training set by applying label-preserving transformations (random crop, horizontal flip, colour jitter, rotation for images).

Mixup (Zhang et al. 2018) interpolates both inputs and soft labels:

$$\hat{x} = \lambda x_i + (1 - \lambda)x_j, \quad \hat{y} = \lambda y_i + (1 - \lambda)y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha).$$

Mixup improves generalisation and reduces overconfident predictions.

Chapter 19

Convolutional Neural Networks

19.1 The Convolution Operation

Definition 19.1.1: 2-D Convolution Layer

Given input volume $H \times W \times C_{\text{in}}$, a filter bank of K filters each of size $F \times F \times C_{\text{in}}$ produces an output volume of size $H' \times W' \times K$:

$$H' = \frac{H - F + 2P}{S} + 1, \quad W' = \frac{W - F + 2P}{S} + 1,$$

with padding P and stride S .

Key properties:

- **Weight sharing:** the same filter applies at every spatial location — a strong inductive bias for translation equivariance.
- **Local connectivity:** each output neuron depends only on an $F \times F$ region, giving sparse connections vs. fully-connected.
- **Depth stacking:** n conv layers of size F give effective receptive field of $n(F - 1) + 1$.

19.2 Pooling

Definition 19.2.1: Max Pooling

Partitions the spatial dimensions into non-overlapping $F \times F$ cells and outputs the maximum in each cell. Reduces spatial resolution by F , providing translational invariance and reducing computation.

19.3 Typical CNN Architecture

$$[\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}]^{\times N} \rightarrow \text{Flatten} \rightarrow [\text{FC} \rightarrow \text{ReLU}]^{\times M} \rightarrow \text{FC} \rightarrow \text{Softmax}.$$

Early layers detect low-level features (edges, colours); deeper layers compose these into high-level semantic concepts.

19.4 Classic Architectures

LeNet-5 (LeCun 1998): earliest successful deep CNN for digit recognition. Architecture on 32×32 input: $C1(6@28^2) \rightarrow S2(6@14^2) \rightarrow C3(16@10^2) \rightarrow S4(16@5^2) \rightarrow C5(120) \rightarrow F6(84) \rightarrow \text{Output}(10)$.

VGGNet (Simonyan & Zisserman 2014): stacks of 3×3 convolutions and 2×2 max-pooling. Three 3×3 layers give the same 7×7 receptive field as one 7×7 filter but with fewer parameters and more non-linearities. VGG-16 has 138M parameters.

GoogLeNet / InceptionNet (Szegedy et al. 2014): *Inception modules* apply 1×1 , 3×3 , and 5×5 convolutions and 3×3 max-pooling in parallel and concatenate along the channel dimension. 1×1 convolutions reduce channel dimensions cheaply (*bottleneck*). Only 5M parameters.

ResNet (He et al. 2016): introduces *residual (skip) connections* to solve the degradation problem — deeper plain networks having higher training error than shallower ones.

Definition 19.4.1: Residual Block

$$H(x) = F(x) + x,$$

where $F(x)$ is the residual to be learned. If the identity mapping is optimal, F is driven to zero. The skip connection provides a gradient highway to earlier layers, enabling networks of 50–152+ layers.

Proposition 19.4.2

Bottleneck Residual Block

For ResNet-50/101/152 the block uses: 1×1 conv (reduce channels) $\rightarrow 3 \times 3$ conv $\rightarrow 1 \times 1$ conv (restore channels), achieving similar complexity to a 3×3 two-layer block but at greater depth.

Chapter 20

Object Detection and Segmentation

20.1 From Classification to Localisation

Image classification assigns one label to the whole image. **Object detection** simultaneously localises all objects with *bounding boxes* and classifies each one.

Definition 20.1.1: Bounding Box and IoU

A bounding box is parametrised as (x_c, y_c, w, h) . Detection quality is measured by Intersection over Union:

$$\text{IoU}(\hat{B}, B^*) = \frac{|\hat{B} \cap B^*|}{|\hat{B} \cup B^*|}.$$

A prediction is a true positive when $\text{IoU} \geq 0.5$ (PASCAL VOC) or averaged over thresholds 0.5–0.95 (COCO mAP).

20.2 Two-Stage Detectors: Faster R-CNN

A **Region Proposal Network (RPN)** slides a small convolutional head over the backbone feature map, predicting objectness scores and bounding-box offsets for a set of *anchors* (fixed-scale, fixed-ratio boxes). Proposals with high objectness are passed to an ROI-pooling layer that extracts fixed-size feature vectors, which are then classified and refined by detection heads. Faster R-CNN achieves high accuracy but requires two forward passes (RPN + detection).

20.3 One-Stage Detectors: YOLO

YOLO (Redmon et al. 2016) divides the image into an $S \times S$ grid; each cell predicts B bounding boxes with confidence scores and C class probabilities in a single forward pass, enabling real-time detection.

20.4 Semantic Segmentation

Definition 20.4.1: Encoder–Decoder Segmentation

Full per-pixel classification. An encoder (convolutional backbone) downsamples the feature map; a decoder upsamples via transposed convolutions to the original resolution. *Skip connections* from encoder to decoder preserve fine spatial detail. U-Net is the canonical example.

Chapter 21

Recurrent Neural Networks

21.1 Sequence Modelling

Many tasks involve variable-length sequential inputs and outputs. Standard feedforward networks require fixed-size inputs; RNNs process sequences by maintaining a *hidden state* that summarises the history.

Definition 21.1.1: Sequence Topologies

- **Many-to-one**: whole sequence $\rightarrow$ single output (e.g. sentiment classification).
- **One-to-many**: single input $\rightarrow$ sequence (e.g. image captioning).
- **Many-to-many (aligned)**: per-step prediction (e.g. video labelling).
- **Seq2seq**: many-to-one encoder compresses input; one-to-many decoder generates output (e.g. machine translation).

21.2 Vanilla RNN

Definition 21.2.1: Elman RNN

At each time step t , with input $x_t \in \mathbb{R}^d$ and previous hidden state $h_{t-1} \in \mathbb{R}^h$:

$$\begin{aligned}h_t &= \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h), \\ y_t &= W_{hy} h_t + b_y.\end{aligned}$$

The **same** weight matrices W_{hh}, W_{xh}, W_{hy} are reused at every step (*weight sharing across time*).

21.3 Backpropagation Through Time

Definition 21.3.1: BPTT

The gradient of the total loss $J_T = \sum_{t=1}^T L_t$ with respect to shared weights is summed over time steps:

$$\frac{\partial J_T}{\partial W} = \sum_{i=1}^T \frac{\partial J_T}{\partial W} \Big|_{(i)}.$$

The gradient at step T with respect to an earlier hidden state h_t requires the product

$$\frac{\partial J_T}{\partial h_t} = \frac{\partial J_T}{\partial h_T} W_{hh}^{T-t} \prod_{t < i \leq T} \text{diag}[\tanh'(W_{hh}h_{i-1} + W_{xh}x_i)].$$

Proposition 21.3.2

Vanishing and Exploding Gradients

The factor W_{hh}^{T-t} causes the gradient to:

- **Vanish:** when singular values of W_{hh} are < 1 — early time steps receive negligible gradient; no long-range learning.
- **Explode:** when singular values of W_{hh} are > 1 — training diverges.

Fix for exploding gradients: **gradient clipping** — if $\|g\| \geq \tau$, set $g \leftarrow (\tau / \|g\|)g$. Fix for vanishing gradients: **gated architectures** (LSTM, GRU).

21.4 Long Short-Term Memory

Definition 21.4.1: LSTM (Hochreiter & Schmidhuber 1997)

An LSTM maintains a *cell state* C_t (long-term memory) and a *hidden state* H_t (short-term memory). At each step, given the concatenation $[H_{t-1}; X_t]$:

$$\begin{aligned} F_t &= \sigma(W_F[H_{t-1}; X_t] + b_F) && \text{(forget gate)} \\ I_t &= \sigma(W_I[H_{t-1}; X_t] + b_I) && \text{(input gate)} \\ \tilde{C}_t &= \tanh(W_C[H_{t-1}; X_t] + b_C) && \text{(candidate cell)} \\ C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t && \text{(cell update)} \\ O_t &= \sigma(W_O[H_{t-1}; X_t] + b_O) && \text{(output gate)} \\ H_t &= O_t \odot \tanh(C_t) && \text{(hidden state)} \end{aligned}$$

The cell state flows through time with only element-wise operations (forget + add), providing a gradient highway and enabling learning of dependencies across hundreds of steps.

Remark.

GRU (Cho et al. 2014) merges cell and hidden state into one vector and uses two gates (update gate z_t , reset gate r_t), matching LSTM performance with fewer parameters.

21.5 Bidirectional and Multi-Layer RNNs

Definition 21.5.1: Bidirectional RNN

Two RNNs process the sequence in forward and backward directions; hidden states are concatenated at each step:

$$\vec{h}_t = \text{RNN}_{\text{FW}}(\vec{h}_{t-1}, x_t), \quad \overleftarrow{h}_t = \text{RNN}_{\text{BW}}(\overleftarrow{h}_{t+1}, x_t), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t].$$

Each position's representation incorporates both past and future context. Transformers largely supersede bidirectional RNNs due to parallelism.

Multi-layer RNNs feed the hidden states of layer ℓ as inputs to layer $\ell + 1$, enabling hierarchical temporal feature extraction.

Chapter 22

Attention and Transformers

22.1 Scaled Dot-Product Attention

Definition 22.1.1: Attention

Given a query $q \in \mathbb{R}^{d_k}$, keys $K \in \mathbb{R}^{n \times d_k}$, and values $V \in \mathbb{R}^{n \times d_v}$:

$$\text{Attention}(q, K, V) = \text{softmax}\left(\frac{qK^\top}{\sqrt{d_k}}\right)V.$$

The factor $1/\sqrt{d_k}$ prevents dot products from saturating the softmax.

In **self-attention**, queries, keys, and values are all derived from the same input sequence X via learned projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Every token attends to every other token simultaneously in $O(1)$ layers, unlike RNNs which require $O(T)$ sequential steps.

22.2 Multi-Head Attention

Definition 22.2.1: Multi-Head Attention

Run h independent attention heads in parallel, each with projections $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$, then concatenate and project:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O, \quad \text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i).$$

Different heads can capture syntactic, semantic, and positional relationships simultaneously.

22.3 The Transformer Architecture

Definition 22.3.1: Transformer (Vaswani et al. 2017)

An encoder–decoder architecture without recurrence.

Encoder block (repeated N times):

1. Multi-head self-attention $\rightarrow$ Add & LayerNorm.
2. Position-wise FFN $\rightarrow$ Add & LayerNorm.

Decoder block (repeated N times):

1. Masked multi-head self-attention (causal; prevents attending to future) $\rightarrow$ Add & LayerNorm.
2. Multi-head cross-attention (queries from decoder; keys, values from encoder) $\rightarrow$ Add & LayerNorm.
3. Position-wise FFN $\rightarrow$ Add & LayerNorm.

Feed-Forward sublayer:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

Layer Normalisation: for each token h_k normalise over the feature dimension, then apply learned scale γ and bias β :

$$\text{LN}(h_k) = \frac{h_k - \mu_k}{\sqrt{\sigma_k + \varepsilon}} \cdot \gamma + \beta.$$

Definition 22.3.2: Positional Encoding

Since Transformers have no recurrence, token order is injected by adding sinusoidal positional embeddings to the input:

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right).$$

Alternatively, learned positional embeddings are used in BERT/GPT.

22.4 Pre-Trained Transformers

BERT (Devlin et al. 2019) is a bidirectional Transformer encoder pre-trained with two self-supervised tasks:

- **Masked Language Modelling (MLM):** randomly mask 15% of tokens; predict the masked tokens.
- **Next Sentence Prediction (NSP):** binary prediction of whether two sentences are consecutive.

BERT-base: 12 layers, hidden size 768, 12 attention heads, 110M parameters. BERT-large: 24 layers, hidden size 1024, 16 heads, 340M parameters. Training corpus: Wikipedia (2.5B words) + BookCorpus (0.8B words). Fine-tuning: attach a task head to the pre-trained encoder; update all parameters on labelled downstream data (*pretrain once, fine-tune many times*).

GPT (Radford et al. 2018) uses a causal (left-to-right) Transformer *decoder* pre-trained with standard language modelling:

$$L = \frac{1}{T} \sum_{t=1}^T L_{\text{CE}}(y_t, \hat{y}_t).$$

GPT-3 (Brown et al. 2020): 175B parameters, trained on 300B tokens. It exhibits **in-context learning**: given task demonstrations in the prompt, it generalises with no gradient updates (a purely forward-pass phenomenon).

Remark.

- **Transformer family.** Encoder-only (BERT, RoBERTa): bidirectional context; excel at classification and NLU.
- Decoder-only (GPT, PaLM): auto-regressive; natural language generation.
- Encoder-decoder (T5, BART): seq2seq tasks (translation, summarisation).

Part V

Generative Models

Chapter 23

Variational Autoencoders

23.1 The Generative Modelling Goal

A **generative model** learns the data distribution $p(x)$ so that one can (i) evaluate likelihood and (ii) draw new samples $x \sim p(x)$. Standard autoencoders compress data but impose no structure on the latent space, making the generative direction unreliable.

23.2 Evidence Lower Bound

Definition 23.2.1: VAE (Kingma & Welling 2014)

Assume a latent variable model $p_\theta(x, z) = p_\theta(x|z)p(z)$ with Gaussian prior $p(z) = \mathcal{N}(0, I)$. The marginal $p_\theta(x) = \int p_\theta(x|z)p(z) dz$ is intractable. Introducing an approximate posterior $q_\phi(z|x)$, the *Evidence Lower Bound* is:

$$\log p_\theta(x) \geq \mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(z|x) \parallel p(z))}_{\text{regularisation}}.$$

Proposition 23.2.2

ELBO Derivation

By Jensen's inequality applied to the concave log:

$$\log p_\theta(x) = \log \mathbb{E}_{q_\phi} \left[\frac{p_\theta(x|z)p(z)}{q_\phi(z|x)} \right] \geq \mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(x|z)p(z)}{q_\phi(z|x)} \right] = \mathcal{L}.$$

The gap is $D_{\text{KL}}(q_\phi(z|x) \parallel p_\theta(z|x)) \geq 0$.

23.3 The Reparametrisation Trick

Sampling $z \sim q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x)I)$ is non-differentiable. Reparametrise:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Randomness is now in ε (no parameters), so gradients flow through μ_ϕ and σ_ϕ via standard backpropagation.

Remark.

Closed-form KL (diagonal Gaussian): $D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$.

Chapter 24

Generative Adversarial Networks

24.1 The GAN Minimax Game

Definition 24.1.1: GAN (Goodfellow et al. 2014)

Train two networks adversarially:

- **Generator** $G_\theta : \mathcal{Z} \rightarrow \mathcal{X}$: maps noise $z \sim p(z)$ to fake samples.
- **Discriminator** $D_\phi : \mathcal{X} \rightarrow [0, 1]$: classifies real vs. generated.

Minimax objective:

$$\min_{\theta} \max_{\phi} \mathbb{E}_{x \sim p_{\text{data}}} [\log D_{\phi}(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D_{\phi}(G_{\theta}(z)))].$$

Proposition 24.1.2

Optimal Discriminator and Equilibrium

At fixed G , the optimal discriminator is $D^*(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_G(x))$. Substituting, the minimax problem reduces to minimising the Jensen–Shannon divergence: $\min_{\theta} 2 D_{\text{JS}}(p_{\text{data}} \| p_G) - \log 4$. Global minimum $p_G = p_{\text{data}}$.

24.2 Practical Variants

Training alternates: update D to distinguish real from fake, then update G to fool D (using the non-saturating loss $-\mathbb{E}[\log D_{\phi}(G_{\theta}(z))]$ to avoid early vanishing gradients).

WGAN (Arjovsky et al. 2017) uses the Wasserstein-1 distance and a 1-Lipschitz critic. With gradient penalty:

$$L_{\text{GP}} = \mathbb{E}[D(\tilde{x})] - \mathbb{E}[D(x)] + \lambda \mathbb{E}[(\|\nabla_{\tilde{x}} D(\hat{x})\|_2 - 1)^2].$$

Conditional GAN: condition both G and D on auxiliary label y to control the class or attributes of generated samples.

Chapter 25

Diffusion Models

25.1 Forward Diffusion Process

Definition 25.1.1: DDPM (Ho et al. 2020)

A fixed *forward process* gradually adds Gaussian noise over T steps:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

Using $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$, the marginal is:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I),$$

so $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$.

25.2 Reverse Process and Training

A neural network $\varepsilon_\theta(x_t, t)$ predicts the noise. The simplified training loss is:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} \left[\left\| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t) \right\|_2^2 \right].$$

To generate: start from $x_T \sim \mathcal{N}(0, I)$ and iterate

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

Remark.

Classifier-Free Guidance: train $\varepsilon_\theta(x_t, t, c)$ jointly with the unconditional model (randomly drop c during training). At inference, amplify the conditioning signal: $\tilde{\varepsilon} = \varepsilon_\theta(x_t, t) + w(\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t))$ with guidance scale $w > 1$. Higher w trades sample diversity for prompt fidelity (used in Stable Diffusion, DALL-E 2).

Part VI

Large Language Models

Chapter 26

Language Models and Pretraining

26.1 Language Modelling

Definition 26.1.1: Language Model

A language model assigns probabilities to token sequences via the chain rule:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

Training minimises cross-entropy (equivalently, maximises log-likelihood):

$$L = -\frac{1}{|\mathcal{D}|} \sum_t \log P_\theta(x_t \mid x_{<t}).$$

Perplexity: $\text{PPL} = \exp(L)$. Lower is better.

26.2 Subword Tokenisation

Raw text is mapped to integers via a subword tokeniser. **Byte-Pair Encoding (BPE)**: iteratively merge the most frequent adjacent byte/character pair, building a vocabulary of $|V|$ subword units (typically 30k–100k).

26.3 Transformer LM Architectures

- **Causal (decoder-only)** — GPT family: masked self-attention; trained on next-token prediction; natural text generation.
- **Masked (encoder-only)** — BERT family: bidirectional attention; trained on masked token prediction; strong NLU representations.
- **Encoder-decoder** — T5, BART: bidirectional encoder plus causal decoder; suited to seq2seq tasks.

26.4 Scaling Laws

Proposition 26.4.1

Chinchilla Scaling Law (Hoffmann et al. 2022)

Loss improves as a power law in model size N and training tokens D :

$$L(N, D) \approx \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_\infty, \quad \alpha \approx \beta \approx 0.5.$$

For compute budget $C \approx 6ND$, the optimal allocation satisfies $N^* \approx D^* \propto \sqrt{C}$: a smaller model trained on more tokens outperforms a larger model trained on fewer.

Chapter 27

Instruction Finetuning and Alignment

27.1 Supervised Fine-Tuning

Pre-trained LLMs predict the next token but do not inherently follow instructions. **Instruction fine-tuning** trains on curated (prompt, response) pairs with standard cross-entropy, teaching the model to be helpful, harmless, and honest.

27.2 Reinforcement Learning from Human Feedback

Definition 27.2.1: RLHF (Christiano et al. 2017; Ouyang et al. 2022)

1. **Reward model**: rank model outputs; train $r_\phi(x, y)$ via

$$L_{\text{RM}} = -\mathbb{E}_{(x, y_w \succ y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))].$$

2. **RL fine-tuning** with PPO: maximise expected reward subject to a KL penalty that keeps the policy close to the SFT reference:

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} [r_\phi(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}).$$

27.3 Direct Preference Optimisation

Definition 27.3.1: DPO (Rafailov et al. 2023)

Bypasses explicit reward model training. The DPO loss:

$$L_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right].$$

DPO is simpler, more stable, and often matches or exceeds RLHF quality.

Chapter 28

In-Context Learning and Prompting

28.1 In-Context Learning

Definition 28.1.1: In-Context Learning (Brown et al. 2020)

Feed the model a prompt containing k demonstrations $(x_1, y_1), \dots, (x_k, y_k)$ then a query x_{k+1} ; the model predicts y_{k+1} via a **single forward pass** — no gradient updates. Zero-shot: no demonstrations. Few-shot: $k \geq 1$.

28.2 Chain-of-Thought Prompting

Chain-of-Thought (CoT) (Wei et al. 2022): include step-by-step reasoning in few-shot exemplars. Dramatically improves performance on arithmetic, commonsense, and symbolic reasoning.

Zero-Shot CoT: append “Let’s think step by step.” to the prompt; elicits reasoning without examples.

Self-Consistency: sample k reasoning chains independently; take the majority-vote final answer. Reduces variance and improves accuracy.

Remark.

Least-to-Most Prompting: decompose a complex problem into easier subproblems; solve each sequentially, passing prior solutions into the next context. Enables compositional generalisation beyond training length.

Chapter 29

Decoding Strategies

29.1 Deterministic Decoding

Definition 29.1.1: Greedy Decoding

$x_t = \arg \max_v P_\theta(v \mid x_{<t})$. Fast but locally optimal; can produce repetitive or suboptimal sequences.

Definition 29.1.2: Beam Search

Maintain B (*beam width*) partial hypotheses; expand each by all vocabulary tokens; keep top- B by cumulative log-probability. Better for structured tasks (translation, summarisation); tends to produce generic text for open-ended generation.

29.2 Stochastic Decoding

Definition 29.2.1: Temperature Sampling

Sample from $P_T(v) \propto P(v)^{1/T}$. $T < 1$: peakier (more greedy); $T > 1$: flatter (more random).

Definition 29.2.2: Top- k Sampling

Restrict to the k highest-probability tokens, renormalise, sample.

Definition 29.2.3: Nucleus (Top- p) Sampling (Holtzman et al. 2020)

Restrict to the smallest vocabulary subset whose cumulative probability $\geq p$ (e.g. $p = 0.9$), renormalise, sample. The set size adapts dynamically: large when the distribution is flat, small when peaked.

Chapter 30

Retrieval-Augmented Generation

30.1 Motivation

LLM parameters encode knowledge frozen at training time and can hallucinate. **RAG** (Lewis et al. 2020) augments the LLM with a dynamically retrieved, up-to-date knowledge base.

Definition 30.1.1: RAG Pipeline

1. **Index:** encode all documents with a dense encoder; store embeddings in a vector database (FAISS, HNSW).
2. **Retrieve:** embed the query q ; find top- k passages by maximum inner product: $\text{score}(q, d) = E_Q(q)^\top E_P(d)$.
3. **Generate:** concatenate query and retrieved passages as context; generate the answer with the LLM.

Definition 30.1.2: Dense Passage Retrieval (Karpukhin et al. 2020)

Train dual encoders E_Q, E_P with in-batch negatives:

$$L = -\log \frac{\exp(E_Q(q)^\top E_P(d^+))}{\exp(E_Q(q)^\top E_P(d^+)) + \sum_j \exp(E_Q(q)^\top E_P(d_j^-))}.$$

Chapter 31

LLM Agents

31.1 Agent Architecture

An **LLM agent** uses a language model as a reasoning engine that perceives observations, maintains memory, plans, and executes actions via external tools (web search, code interpreter, databases, APIs).

Definition 31.1.1: ReAct Framework (Yao et al. 2023)

Interleave *Reasoning* (Thought) and *Acting* (Action) steps:

1. **Thought:** the LLM generates a natural-language reasoning step.
2. **Action:** call a tool (search, calculator, code runner).
3. **Observation:** the tool returns a result.
4. Repeat until a final answer is produced.

The reasoning trace makes the agent's decision process interpretable and debuggable.

31.2 Memory Types

- **In-context:** past conversation and retrieved documents within the prompt window.
- **External:** vector database of past experiences retrieved by embedding similarity.
- **Parametric:** knowledge encoded in model weights at pretraining.

31.3 Multi-Agent Systems

A *planner* agent decomposes a task into subtasks; specialised *worker* agents execute each; a *critic* agent reviews and flags errors. Communication is mediated through shared message queues or structured prompts (e.g. AutoGen, CrewAI frameworks).

Part VII

ML Engineering

Chapter 32

Transfer Learning and Fine-Tuning

32.1 Transfer Learning

Definition 32.1.1: Transfer Learning

Pre-train on a large source task; adapt to a target task with limited labelled data.

- **Feature extraction:** freeze pre-trained weights; train only a new task head. Fast; best when source and target are similar.
- **Full fine-tuning:** continue training all weights on the target dataset with a small learning rate. Better when sufficient target data is available.

32.2 Parameter-Efficient Fine-Tuning

Fine-tuning billion-parameter models is expensive. PEFT methods adapt a small parameter subset.

Definition 32.2.1: LoRA (Hu et al. 2022)

Constrain the weight update ΔW to a low-rank factorisation:

$$\Delta W = AB^\top, \quad A \in \mathbb{R}^{m \times r}, \quad B \in \mathbb{R}^{n \times r}, \quad r \ll \min(m, n).$$

Only A and B are trained (W_0 frozen), reducing trainable parameters from mn to $r(m + n)$. LoRA adds negligible inference latency when merged: $W = W_0 + AB^\top$.

Prompt tuning (Lester et al. 2021): prepend a small set of learnable continuous “soft prompt” vectors to the input embeddings; freeze all other parameters. Competitive with full fine-tuning at large model size.

Chapter 33

Hyperparameter Optimization

33.1 Problem Statement

Definition 33.1.1: HPO

Find $\lambda^* = \arg \min_{\lambda \in \Lambda} f(\lambda)$ where $f(\lambda)$ is validation loss after training with hyperparameters λ (learning rate, architecture width/depth, regularisation, batch size). Each evaluation requires a complete training run — expensive black-box optimisation.

33.2 Search Strategies

Grid search: exhaustive enumeration of a discrete grid. Exponential in the number of hyperparameters; impractical beyond 3–4 dimensions.

Random search (Bergstra & Bengio 2012): sample λ uniformly at random. More efficient when only a few hyperparameters are important.

Definition 33.2.1: Bayesian Optimisation

Build a probabilistic surrogate model of f (Gaussian process or Tree Parzen Estimator). Select the next configuration by maximising an *acquisition function*, e.g. Expected Improvement:

$$\text{EI}(\lambda) = \mathbb{E}[\max(f^* - f(\lambda), 0)].$$

After each evaluation, update the surrogate. Achieves sample efficiency superior to random search for moderate-dimensional, expensive objectives.

33.3 Multi-Fidelity Methods

Successive Halving: run n configurations for a small budget; keep the top half; double the budget; repeat until one remains.

Hyperband: run successive halving with several different initial bracket sizes in parallel to balance exploration and exploitation.

Chapter 34

Distributed Training

34.1 Parallelism Strategies

Data parallelism: replicate the model on each device; partition the mini-batch across devices; synchronise gradients via all-reduce. Preferred when the model fits on one device.

Pipeline parallelism (GPipe): split layers into stages across devices; pass micro-batches through the pipeline to keep all stages busy.

Tensor parallelism (Megatron-LM): split individual weight matrices across devices; each computes a partial matrix product and results are summed by all-reduce. Required for very large layers.

34.2 Gradient Synchronisation

Definition 34.2.1: Ring All-Reduce

In a ring of N workers, each sends and receives $1/N$ of the gradient tensor per step. Total communication $2(N-1)/N |\theta|$. Bandwidth-optimal: the bottleneck does not grow with N .

34.3 Mixed-Precision Training

Store master weights in FP32 but compute forward/backward passes in FP16 or BF16. Use *loss scaling* to prevent gradient underflow. Halves memory and doubles throughput on Tensor Core hardware.

Chapter 35

AutoML and Knowledge Distillation

35.1 Neural Architecture Search

Definition 35.1.1: NAS

Automatically search a discrete space of architectures (layer types, widths, skip connections) to maximise validation performance.

- **RL-based:** a controller RNN proposes architectures as actions; validation accuracy is the reward.
- **DARTS** (Liu et al. 2019): relax discrete choices to a softmax mixture; optimise mixing weights α by gradient descent on validation loss; discretise at the end.

35.2 Knowledge Distillation

Definition 35.2.1: Knowledge Distillation (Hinton et al. 2015)

Transfer knowledge from a large *teacher* T to a smaller *student* S by matching soft output distributions:

$$L_{\text{KD}} = (1 - \alpha) L_{\text{CE}}(y, \hat{y}_S) + \alpha \tau^2 D_{\text{KL}}\left(\text{softmax}_{\frac{z_T}{\tau}} \parallel \text{softmax}_{\frac{z_S}{\tau}}\right),$$

where temperature $\tau > 1$ softens the teacher's distribution, revealing inter-class similarity structure not present in hard labels.

Part VIII

Reinforcement Learning

Chapter 36

Multi-Armed Bandits

36.1 The Bandit Problem

Definition 36.1.1: K -Armed Bandit

At each round t , an agent selects arm $A_t \in \{1, \dots, K\}$ and receives stochastic reward $R_t \sim \mathcal{D}_{A_t}$ with unknown mean μ_a . Cumulative regret over T rounds:

$$\text{Regret}(T) = T\mu^* - \mathbb{E} \left[\sum_{t=1}^T R_t \right], \quad \mu^* = \max_a \mu_a.$$

36.2 Exploration–Exploitation Strategies

ε -greedy: with probability ε pull a random arm; otherwise pull the empirically best arm. Simple but suboptimal.

Definition 36.2.1: UCB1 (Auer et al. 2002)

$$A_t = \arg \max_a \left(\hat{\mu}_a + \sqrt{\frac{2 \ln t}{N_a}} \right),$$

where N_a is the pull count for arm a . The confidence bonus drives exploration of under-tried arms. UCB1 achieves $O(\ln T)$ regret.

Definition 36.2.2: Thompson Sampling

Maintain posterior $P(\mu_a | \text{history})$. Each round, sample $\tilde{\mu}_a \sim P(\mu_a | \cdot)$ and pull $\arg \max_a \tilde{\mu}_a$. With Beta–Bernoulli conjugacy this achieves near-optimal expected regret and is empirically robust.

Chapter 37

Markov Decision Processes

37.1 MDP Framework

Definition 37.1.1: Markov Decision Process

A tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: state space, action space, transition $P(s'|s, a)$, expected reward $R(s, a)$, discount $\gamma \in [0, 1)$. The agent follows policy $\pi(a|s)$ maximising expected discounted return $G_t = \sum_{k \geq 0} \gamma^k R_{t+k+1}$.

Definition 37.1.2: Value Functions

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k R_{t+k+1} \mid S_t = s \right],$$
$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k \geq 0} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

37.2 Bellman Equations

Proposition 37.2.1

Bellman Expectation Equations

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^\pi(s')],$$
$$Q^\pi(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right].$$

Proposition 37.2.2

Bellman Optimality Equations

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^*(s')],$$
$$Q^*(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma \max_{a'} Q^*(s', a')].$$

Optimal policy: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Chapter 38

Dynamic Programming

38.1 Policy Evaluation and Improvement

Definition 38.1.1: Iterative Policy Evaluation

Initialise $V_0 \equiv 0$. Iterate:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')].$$

Converges to V^π (contraction mapping in ℓ_∞).

Definition 38.1.2: Policy Improvement

The greedy policy $\pi'(s) = \arg \max_a Q^\pi(s, a)$ satisfies $V^{\pi'}(s) \geq V^\pi(s)$ for all s (*Policy Improvement Theorem*).

Definition 38.1.3: Policy Iteration

Alternate policy evaluation and policy improvement until convergence. Terminates in finitely many steps for finite MDPs.

Definition 38.1.4: Value Iteration

Directly apply the Bellman optimality operator:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')].$$

Converges to V^* without a separate policy evaluation step.

Chapter 39

Temporal Difference Learning

39.1 TD Prediction

Definition 39.1.1: TD(0)

Update the value estimate after each transition $(S_t, A_t, R_{t+1}, S_{t+1})$:

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD target}}.$$

Bootstraps from current estimates (unlike Monte Carlo which waits for episode end); can learn online and in continuing tasks.

39.2 Q-Learning and SARSA

Definition 39.2.1: Q-Learning (Watkins 1989)

Off-policy TD control:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)].$$

Converges to Q^* regardless of the behaviour policy (off-policy).

Definition 39.2.2: SARSA

On-policy TD control using the *actual* next action A_{t+1} :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Converges to Q^{π_ϵ} (the ϵ -greedy policy).

39.3 Deep Q-Network

Definition 39.3.1: DQN (Mnih et al. 2015)

Approximate Q^* with a deep network $Q_\theta(s, a)$. Two stabilisation techniques:

- **Experience replay:** store (s, a, r, s') in a replay buffer; sample random mini-batches to break temporal correlations.
- **Target network:** a periodically-frozen copy Q_{θ^-} provides stable TD targets:

$$L(\theta) = \mathbb{E} \left[\left(R + \gamma \max_{a'} Q_{\theta^-}(S', a') - Q_\theta(S, A) \right)^2 \right].$$

Chapter 40

Policy Gradient Methods

40.1 Policy Gradient Theorem

Definition 40.1.1: REINFORCE (Williams 1992)

Parametrise $\pi_\theta(a|s)$ directly. The policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(A_t|S_t) \cdot G_t \right].$$

Unbiased but high variance; reduces variance by subtracting a baseline b : $\hat{A}_t = G_t - b(S_t)$ (the *advantage*).

40.2 Actor-Critic

Definition 40.2.1: Actor-Critic

Maintain two networks:

- **Actor** π_θ : updated by policy gradient using the TD advantage $\hat{A}_t = R_{t+1} + \gamma V_\phi(S_{t+1}) - V_\phi(S_t)$.
- **Critic** V_ϕ : updated by TD learning.

The critic's bootstrap estimate reduces variance vs. REINFORCE while remaining lower bias than using a fully-learned value function.

40.3 Proximal Policy Optimisation

Definition 40.3.1: PPO (Schulman et al. 2017)

Clip the policy ratio to prevent large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right],$$

where $r_t(\theta) = \pi_\theta(A_t|S_t)/\pi_{\theta_{\text{old}}}(A_t|S_t)$. Clipping prevents the new policy from deviating too far from the old one. PPO is the dominant RL algorithm used for RLHF alignment of LLMs.